Université Hassan II de Casablanca

Faculté des Sciences Ben M'Sick

Département de Mathématiques et Informatique

Master d'Excellence en Intelligence Artificielle

Rapport de Projet Final

Mise en place d'une infrastructure DevOps

avec CI/CD, Kubernetes et Monitoring

Application Myway – Gestion de Modules Étudiants

Réalisé par :

Anass Dabibe

Adnane Dahbi

Yassine Farih

Youssef Sarraf

Encadré par :

Pr. Fouzia Benabbou

Module :

Cloud Computing & DevOps

Année Universitaire : 2025 – 2026

Remerciements

Au terme de ce travail, nous tenons en premier lieu à exprimer notre profonde gratitude à notre encadrante, **Pr. Fouzia Benabbou**. Ses directives précieuses, sa rigueur scientifique et ses encouragements constants ont été les piliers de la réussite de ce projet.

Nos remerciements s'adressent également à l'ensemble du corps professoral et administratif du **Master d'Excellence en Intelligence Artificielle** de la **Faculté des Sciences Ben M'Sick**, pour la qualité des enseignements dispensés et pour avoir mis à notre disposition un cadre de travail propice à l'apprentissage.

Enfin, nous dédions nos pensées les plus affectueuses à nos familles respectives et à nos proches. Leur soutien moral infaillible, leur patience et leurs sacrifices ont été notre plus grande source de motivation durant nos études. Que ce travail soit l'expression de notre profonde reconnaissance.

Abstract

This report presents the design and deployment of a complete DevOps infrastructure for the **Myway** application, a student module management platform built with React and Node.js/Express. The project was carried out as part of the Master of Excellence in Artificial Intelligence program at the Faculty of Sciences Ben M'Sick, Hassan II University of Casablanca.

The main objective was to implement an end-to-end automated delivery pipeline, deploying a containerized application on a Kubernetes cluster hosted on AWS, while integrating continuous quality and security controls.

The infrastructure relies on **Terraform** for cloud provisioning (VPC, EC2 instances, Elastic IPs), **Ansible** for automated node configuration (Docker installation, Kubernetes cluster bootstrap via kubeadm v1.29.0), and **GitHub Actions** for the CI/CD pipeline. The pipeline consists of six sequential jobs : unit tests, static code analysis (SonarCloud), Docker image build and push (DockerHub), container security scanning (Trivy), dynamic application security testing (OWASP ZAP), and automated deployment to Kubernetes with rollback capability.

Monitoring is handled by **Prometheus** and **Grafana**, providing real-time visibility into system metrics (CPU, memory, disk) on the worker node.

All steps, from AWS account creation to production deployment, are documented with screenshots and configuration excerpts to enable full reproducibility of the infrastructure.

Keywords : DevOps, CI/CD, Kubernetes, AWS, Terraform, Ansible, Docker, GitHub Actions, SonarCloud, Trivy, OWASP ZAP, Prometheus, Grafana.

Résumé

Ce rapport présente la conception et le déploiement d'une infrastructure DevOps complète pour l'application **Myway**, une plateforme de gestion de modules étudiants développée en React et Node.js. Le projet a été réalisé dans le cadre du Master d'Excellence en Intelligence Artificielle à la Faculté des Sciences Ben M'Sick de l'Université Hassan II de Casablanca.

L'objectif principal était de mettre en oeuvre une chaîne CI/CD automatisée, de déployer l'application sur un cluster Kubernetes hébergé sur AWS, et d'intégrer un système de monitoring. Les technologies suivantes ont été employées :

— **Infrastructure as Code :** Terraform pour le provisionnement des ressources AWS (instances EC2, VPC, sous-réseaux, adresses IP élastiques).

— **Automatisation de la configuration :** Ansible pour l'installation de Docker, Kubernetes et le déploiement des playbooks.

— **Orchestration :** Kubernetes (kubeadm, v1.29.0) avec un noeud maître et un noeud worker en région `eu-west-1` (Irlande).

— **Pipeline CI/CD :** GitHub Actions avec six étapes : tests, analyse de qualité (SonarCloud), build et push des images Docker, scan de sécurité (Trivy), scan DAST (OWASP ZAP) et déploiement automatisé sur Kubernetes.

— **Monitoring :** Prometheus et Grafana pour la supervision des métriques système et applicatives.

L'ensemble de l'infrastructure, de la création du compte AWS jusqu'au déploiement en production, est documenté étape par étape avec des captures d'écran et extraits de configuration.

Mots-clés : DevOps, CI/CD, Kubernetes, AWS, Terraform, Ansible, Docker, GitHub Actions, SonarCloud, Trivy, OWASP ZAP, Prometheus, Grafana.

Table des matières

Remerciements **i**

Abstract **ii**

Résumé **iii**

Table des matières **iv**

Table des figures **vii**

Liste des Abréviations **ix**

1 Introduction **2**
1.1 Contexte général 2
1.2 Problématique et objectifs 2
1.3 Méthodologie et organisation de l'équipe 3
1.4 Choix technologiques justifiés 4
1.5 Organisation du rapport 4

2 Architecture Globale **5**
2.1 Vue d'ensemble 5
2.2 Stack technologique 6
2.3 Principes DevSecOps appliqués 7
2.4 Flux de données et interactions entre composants 7

3 Application Myway **8**
3.1 Description fonctionnelle 8
3.2 Architecture de l'application 8
3.3 API REST du backend 9
3.4 Dockerisation de l'application 9
3.4.1 Dockerfile du backend 9
3.4.2 Dockerfile du frontend 10
3.4.3 Docker Compose 10

4 Infrastructure AWS avec Terraform et Ansible 11
4.1 Création du compte AWS et configuration IAM . . . 11
4.1.1 Création du compte AWS Free Tier . . . 11
4.1.2 Configuration IAM . . . 12
4.1.3 Génération des clés d'accès AWS . . . 15
4.1.4 Configuration de l'AWS CLI . . . 16
4.2 Infrastructure as Code avec Terraform . . . 17
4.2.1 Vérification des prérequis . . . 17
4.2.2 Configuration Terraform . . . 17
4.2.3 Déploiement Terraform . . . 19
4.2.4 Adresses IP Élastiques . . . 20
4.2.5 Instances EC2 en production . . . 23
4.3 Configuration automatisée avec Ansible . . . 24
4.3.1 Inventaire Ansible . . . 24
4.3.2 Playbook 1 : Installation de Docker . . . 25
4.3.3 Playbook 2 : Installation de Kubernetes . . . 26
4.3.4 Playbook 3 : Initialisation du noeud maître . . . 28
4.3.5 Playbook 4 : Jonction du worker au cluster . . . 30
4.3.6 Playbook 5 : Déploiement du Monitoring . . . 31
4.3.7 Playbook 6 : Installation du Dashboard Kubernetes . . . 31

5 Déploiement Kubernetes 32
5.1 Manifests Kubernetes . . . 32
5.1.1 Namespace . . . 32
5.1.2 Déploiement du Backend . . . 32
5.1.3 Déploiement du Frontend . . . 34
5.1.4 Monitoring Kubernetes . . . 35
5.1.5 Horizontal Pod Autoscaling (HPA) . . . 41
5.2 État du cluster Kubernetes . . . 42
5.3 Aperçu de l'application déployée . . . 43

6 Pipeline CI/CD avec GitHub Actions 44
6.1 Vue d'ensemble du pipeline . . . 44
6.2 Configuration du pipeline . . . 44
6.3 Détail des jobs du pipeline . . . 49
6.3.1 Job 1 : Tests Backend et Frontend . . . 49
6.3.2 Job 2 : Analyse de qualité SonarCloud . . . 50
6.3.3 Job 3 : Build et Push des images Docker . . . 51
6.3.4 Job 4 : Scan de sécurité Trivy . . . 52
6.3.5 Job 5 : DAST avec OWASP ZAP . . . 53

6.3.6 Job 6 : Déploiement sur Kubernetes (AWS) 54
6.4 Secrets GitHub Actions configurés 55
6.5 Durée et performances du pipeline 55

7 Monitoring avec Prometheus et Grafana 56
7.1 Architecture du monitoring 56
7.2 Prometheus 57
7.3 Grafana 58

8 Guide de Reproduction de l'Infrastructure 59
8.1 Prérequis 59
8.2 Étapes de déploiement 59
8.2.1 Étape 1 : Cloner le dépôt 59
8.2.2 Étape 2 : Configurer AWS 59
8.2.3 Étape 3 : Déployer l'infrastructure Terraform 60
8.2.4 Étape 4 : Allouer et associer les Elastic IPs 60
8.2.5 Étape 5 : Configurer et exécuter les playbooks Ansible 60
8.2.6 Étape 6 : Configurer les secrets GitHub Actions 61
8.2.7 Étape 7 : Déclencher le pipeline CI/CD 61
8.2.8 Étape 8 : Accéder au Monitoring et au Dashboard 61
8.2.9 Nettoyage des ressources AWS 61

9 Conclusion 62
9.1 Conformité au cahier des charges 62
9.2 Bilan du projet 63
9.3 Compétences acquises 64
9.4 Limites techniques identifiées 64
9.5 Perspectives d'amélioration 64

Bibliographie 65

Table des figures

2.1 Architecture globale de l'infrastructure Myway 5

4.1 Initialisation du compte AWS Free Tier 11
4.2 Console AWS après création du compte 12
4.3 Accès au service IAM depuis la console AWS 12
4.4 Groupe IAM project_devops_team avec 4 membres 13
4.5 Politique AmazonEC2ReadOnlyAccess attachée au groupe 13
4.6 Création d'un utilisateur IAM avec accès console 14
4.7 Attachement de l'utilisateur au groupe project_devops_team 14
4.8 Confirmation de création de l'utilisateur IAM 15
4.9 Page de gestion des credentials AWS (Free Tier actif) 15
4.10 Récupération de la clé d'accès AWS 16
4.11 Vérification de la configuration AWS CLI avec sts get-caller-identity 16
4.12 Versions installées : Terraform v1.14.7, Ansible 2.19.7, AWS CLI 2.34.8, kubectl v1.35.1 17
4.13 Initialisation de Terraform (`terraform init`) 19
4.14 Plan Terraform (`terraform plan`) 19
4.15 Sorties Terraform après `terraform apply` 20
4.16 Navigation vers le service EC2 dans la console AWS 20
4.17 Sélection de la région eu-west-1 (Irlande) 21
4.18 Allocation d'une Elastic IP depuis EC2 21
4.19 Paramètres d'allocation de l'Elastic IP 22
4.20 Association de l'Elastic IP à l'instance k8s-master 22
4.21 Console AWS EC2 – instances k8s-master et k8s-worker en état Running . 23
4.22 Fichier inventory.ini mis à jour avec les IPs élastiques 24
4.23 Test de connectivité Ansible (`ansible all -m ping`) – SUCCESS sur les deux noeuds 25
4.24 Exécution du playbook 01-install-docker.yml – Docker installé sur master et worker 25
4.25 Exécution du playbook 02-install-kubernetes.yml – Kubernetes v1.29.0 installé 27

4.26 Exécution du playbook 03-init-master.yml – Flannel installé, commande join générée 29
4.27 Worker joint au cluster avec succès – état Ready en 6m27s 30
4.28 Vérification finale – les deux noeuds sont Ready (kubectl get nodes) 31

5.1 Tableau de bord Kubernetes – namespace myway : 4 déploiements, 2 pods, 10 replica sets 42
5.2 Interface de l'application Myway en production sur le cluster Kubernetes . 43

6.1 Vue globale du pipeline DevSecOps – 6 jobs réussis, durée totale : 5m 55s . 44
6.2 Job 1 – Tests backend et build frontend réussis en 32s 49
6.3 Job 2 – Analyse SonarCloud réussie en 1m 11s 50
6.4 Job 3 – Images Docker buildées et pushées sur DockerHub en 48s 51
6.5 Job 4 – Scan Trivy des images Docker et du système de fichiers en 48s . . . 52
6.6 Job 5 – Scan DAST OWASP ZAP sur l'API backend en 46s 53
6.7 Job 6 – Déploiement Kubernetes réussi en 1m 28s, rollback non déclenché . 54

7.1 Interface Prometheus – métrique process_cpu_seconds_total 57
7.2 Tableau de bord Grafana – Node Exporter : métriques système du backend Myway 58

Liste des Abréviations

Abréviation	Signification
AMI	Amazon Machine Image
API	Application Programming Interface
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Delivery
CLI	Command Line Interface
CNI	Container Network Interface
CPU	Central Processing Unit
DAST	Dynamic Application Security Testing
DNS	Domain Name System
DockerHub	Registre public d'images Docker
EBS	Elastic Block Store
EC2	Elastic Compute Cloud
EIP	Elastic IP Address
GiB	Gibibyte
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IaC	Infrastructure as Code
IAM	Identity and Access Management
IP	Internet Protocol
JSON	JavaScript Object Notation
K8s	Kubernetes (abréviation numérique)
MiB	Mebibyte
npm	Node Package Manager
OWASP	Open Web Application Security Project

Abréviation	**Signification**
PVC	PersistentVolumeClaim
RAM	Random Access Memory
RDS	Relational Database Service
REST	Representational State Transfer
SARIF	Static Analysis Results Interchange Format
SCA	Software Composition Analysis
SAST	Static Application Security Testing
SSH	Secure Shell
TLS	Transport Layer Security
VPC	Virtual Private Cloud
YAML	YAML Ain't Markup Language
ZAP	Zed Attack Proxy

Chapitre 1

Introduction

1.1 Contexte général

Le mouvement DevOps est né de la nécessité de réconcilier deux équipes historiquement en tension dans les organisations informatiques : les développeurs, qui souhaitent livrer des fonctionnalités rapidement, et les opérations, qui privilégient la stabilité des systèmes. DevOps propose une culture de collaboration, appuyée par des outils et des pratiques d'automatisation, permettant de réduire le temps entre l'écriture d'une ligne de code et son exécution en production tout en maintenant un haut niveau de fiabilité.

Dans ce contexte, le module **Cloud Computing & DevOps** du Master d'Excellence en Intelligence Artificielle de la Faculté des Sciences Ben M'Sick nous a conduit à concevoir et déployer une infrastructure DevOps complète et reproductible pour une application réelle. Au-delà de la simple mise en place d'outils, ce projet vise à comprendre les interdépendances entre chaque brique technologique et à mesurer concrètement les bénéfices d'une chaîne de livraison automatisée.

1.2 Problématique et objectifs

Sans automatisation, le déploiement d'une application web implique une série d'opérations manuelles répétitives et sources d'erreurs : configuration des serveurs, installation des dépendances, mise à jour manuelle des conteneurs, vérification de l'état des services... Ces opérations ne passent pas à l'échelle et introduisent des risques opérationnels importants. Notre projet répond à cette problématique en implémentant quatre grandes familles d'automatisation :

— **Infrastructure as Code (IaC) :** l'ensemble des ressources cloud AWS (machines virtuelles, réseau, sécurité) est décrit dans des fichiers Terraform versionnés. Un seul `terraform apply` recrée l'intégralité de l'infrastructure à l'identique.

— **Configuration Management :** Ansible configure les noeuds du cluster (installation Docker, Kubernetes, paramètres système) de manière idempotente via des playbooks déclaratifs.

— **Intégration et Livraison Continues (CI/CD) :** GitHub Actions orchestre un pipeline de six jobs qui, à chaque `git push`, teste, analyse, sécurise et déploie automatiquement l'application sur le cluster Kubernetes.
— **Observabilité :** Prometheus collecte les métriques système et applicatives ; Grafana les visualise sous forme de tableaux de bord interactifs permettant de détecter anomalies et dégradations de performances.

L'application support de ce projet est **Myway**, une plateforme web destinée aux étudiants pour organiser leurs modules d'études, leurs ressources pédagogiques, leurs tâches prioritaires et leurs sessions de travail (technique Pomodoro). Développée en React (frontend) et Node.js/Express (backend) avec SQLite3, sa relative simplicité fonctionnelle nous a permis de concentrer l'effort sur l'infrastructure plutôt que sur le code métier.

Au-delà de ces quatre piliers classiques, notre équipe a pris l'initiative d'élever ce projet au standard **DevSecOps**. En intégrant des outils d'analyse statique (SAST), d'analyse de composition (SCA) et de tests dynamiques (DAST) directement dans le pipeline CI/CD, nous appliquons le principe du *Shift-Left Security* : la sécurité n'est plus un audit final, mais une exigence continue validée à chaque `git push`.

1.3 Méthodologie et organisation de l'équipe

Pour mener à bien ce projet dans le délai imparti, notre équipe de quatre personnes a adopté une approche collaborative inspirée des méthodes agiles. La répartition du travail s'est structurée autour des différentes couches de l'infrastructure :

— **Développement et conteneurisation :** écriture du code de l'application Myway, création des Dockerfiles optimisés et configuration du `docker-compose` pour les tests locaux.
— **Infrastructure cloud :** conception des scripts Terraform pour la création du VPC, des groupes de sécurité et des instances EC2 sur AWS.
— **Configuration et Orchestration :** rédaction des playbooks Ansible pour l'installation des dépendances et l'initialisation du cluster Kubernetes via `kubeadm`.
— **Intégration et Déploiement continus :** conception du workflow GitHub Actions, intégration des outils DevSecOps et mise en place de l'observabilité (Prometheus/-Grafana).

Le dépôt GitHub a servi de point central pour la collaboration. Chaque modification de l'infrastructure ou du code applicatif faisait l'objet d'un commit documenté, déclenchant le pipeline CI/CD pour valider la non-régression du système.

1.4 Choix technologiques justifiés

Le choix des outils n'a pas été arbitraire. Chaque technologie a été sélectionnée pour des raisons précises :

Table 1.1 – Justification des choix technologiques

Outil	Justification
AWS	Fournisseur cloud leader avec un Free Tier généreux (750h/mois d'instances t3.micro). Richesse des services managés et documentation exhaustive.
Terraform	Standard de facto pour l'IaC multi-cloud. Syntaxe HCL lisible, plan d'exécution explicite (`terraform plan`), gestion de l'état des ressources.
Ansible	Agentless (pas de daemon à installer sur les noeuds), idempotent, lisible en YAML. Idéal pour bootstrapper un cluster Kubernetes sur des instances EC2 vierges.
Kubernetes	Orchestrateur de conteneurs standard industriel. kubeadm permet un déploiement sur mesure sur des VMs EC2 sans surcoût lié à un service managé comme EKS.
GitHub Actions	CI/CD natif au dépôt GitHub. Marketplace riche (SonarCloud, Trivy, ZAP). Pas d'infrastructure CI à maintenir.
Prometheus + Grafana	Stack de monitoring open source de référence. Intégration native avec Kubernetes via Node Exporter.

1.5 Organisation du rapport

Ce rapport est organisé en huit chapitres. Le chapitre 2 présente l'architecture globale et les principes DevSecOps appliqués. Le chapitre 3 décrit l'application Myway et sa conteneurisation. Les chapitres 4 et 5 couvrent respectivement l'infrastructure AWS (Terraform, Ansible) et le déploiement Kubernetes. Le chapitre 6 détaille le pipeline CI/CD dans sa dimension DevSecOps. Le chapitre 7 présente le monitoring. Enfin, le chapitre 8 constitue un guide de reproduction complet de l'ensemble de l'infrastructure.

Chapitre 2

Architecture Globale

2.1 Vue d'ensemble

L'infrastructure Myway suit le modèle **DevSecOps** : Security as Code intégré dès la phase de développement, et non ajouté en fin de chaîne. Elle repose sur une architecture en couches où chaque composant joue un rôle précis dans le cycle de vie de l'application, du commit du développeur jusqu'au déploiement en production et à la supervision continue.

Le diagramme de la page de garde illustre le cycle complet. On distingue cinq grandes couches fonctionnelles : (1) Développement, (2) Pipeline CI/CD avec contrôles qualité et sécurité, (3) Orchestration Kubernetes, (4) Infrastructure Cloud AWS provisionnée par Terraform et Ansible, (5) Monitoring Prometheus/Grafana.

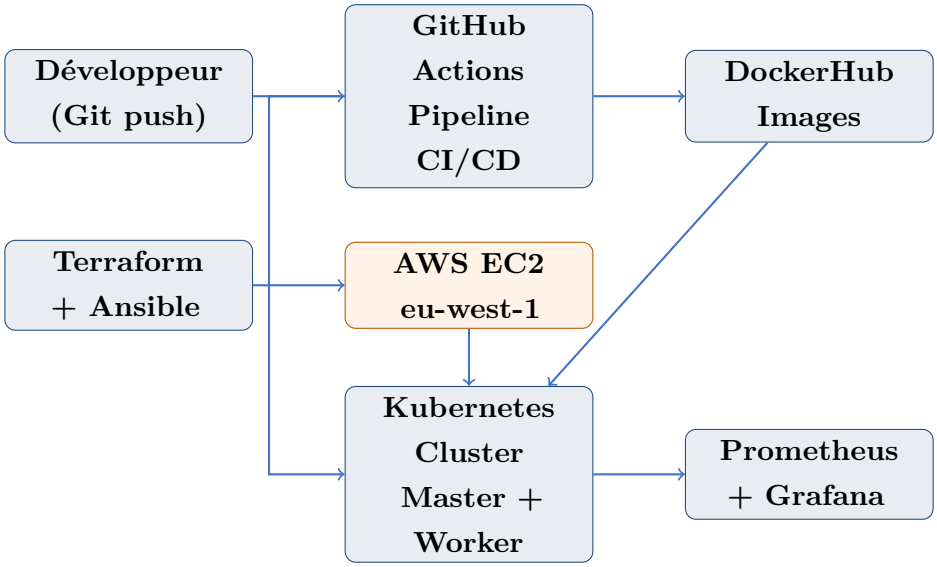

FIGURE 2.1 – Architecture globale de l'infrastructure Myway

2.2 Stack technologique

Table 2.1 – Stack technologique du projet

Catégorie	Outil	Rôle
Application	React + Vite	Interface utilisateur (frontend)
Application	Node.js + Express	API REST (backend)
Application	SQLite3	Base de données embarquée
Conteneurisation	Docker	Packaging de l'application
Registre	DockerHub	Stockage des images (`kappa20/myway-*`)
IaC	Terraform v1.14.7	Provisionnement AWS
Configuration	Ansible core 2.19.7	Déploiement et configuration
Cloud	AWS EC2 (t3.micro)	Machines virtuelles
Région	eu-west-1 (Irlande)	Hébergement
Orchestration	Kubernetes v1.29.0	Gestion des conteneurs
CNI	Flannel	Réseau des pods
CI/CD	GitHub Actions	Pipeline automatisé
Qualité	SonarCloud	Analyse statique du code
Sécurité	Trivy	Scan des images Docker
DAST	OWASP ZAP	Test de sécurité dynamique
Monitoring	Prometheus	Collecte des métriques
Monitoring	Grafana	Tableaux de bord

2.3 Principes DevSecOps appliqués

Ce projet intègre la sécurité à chaque étape du pipeline, conformément aux principes du mouvement **DevSecOps** (Development, Security, Operations) :

TABLE 2.2 – Intégration de la sécurité dans le pipeline (Shift-Left Security)

Phase	Outil	Type de contrôle
Commit	SonarCloud	SAST – détection de bugs, code smells, vulnérabilités dans le code source
Build	Trivy	SCA – scan des images Docker contre les CVE des dépendances système et npm
Test	OWASP ZAP	DAST – simulation d'attaques HTTP sur l'API backend en mode test
Deploy	Kubernetes RBAC + .trivyignore	ClusterRole pour Prometheus ; liste d'exclusions Trivy pour les CVE acceptées
Run	Prometheus	Détection d'anomalies métriques en production

2.4 Flux de données et interactions entre composants

La figure 2.1 montre les relations entre les composants. En voici le flux détaillé :

1. Le développeur fait un `git push` sur la branche `main` du dépôt GitHub `kappa20/Myway`.
2. GitHub Actions déclenche automatiquement le pipeline `cicd.yml`. Les tests unitaires, l'analyse SonarCloud, le build des images Docker et les scans Trivy/OWASP ZAP s'enchaînent.
3. Si tous les jobs réussissent, les images `kappa20/myway-backend:latest` et `kappa20/myway-frontend:latest` sont poussées sur DockerHub.
4. Le job de déploiement se connecte via SSH au noeud maître Kubernetes (instance EC2 `k8s-master`), copie les manifests YAML et exécute `kubectl apply`.
5. Kubernetes orchestre le déploiement sur le noeud worker (`k8s-worker`) : pull des images Docker, création des pods, exposition via NodePort.
6. Prometheus scrape en continu les métriques des pods et du système. Grafana les affiche sur des tableaux de bord accessibles en temps réel.

Ce flux garantit qu'aucun code non testé et non scanné ne peut atteindre l'environnement de production.

Chapitre 3

Application Myway

3.1 Description fonctionnelle

Myway est une application web conçue pour aider les étudiants à organiser leurs études. Elle offre quatre fonctionnalités principales :

- **Gestion des modules :** création et organisation des modules de cours avec des couleurs personnalisées.
- **Gestion des ressources :** ajout d'URLs, de notes et de fichiers attachés à chaque module.
- **Listes de tâches :** création et suivi de tâches prioritaires par module.
- **Minuteur Pomodoro :** timer intégré pour structurer les sessions de travail.

3.2 Architecture de l'application

Table 3.1 – Structure du dépôt GitHub kappa20/Myway

Dossier / Fichier	Description
`frontend/`	Application React (Vite, Context API) – port 3000
`backend/`	API Express.js + SQLite3 – port 5000
`backend/routes/`	Routes API (modules, ressources, todos)
`backend/database/`	Schéma SQLite et seed
`terraform/`	Infrastructure AWS as Code
`ansible/`	Playbooks de configuration
`k8s/`	Manifests Kubernetes
`.github/workflows/`	Pipeline GitHub Actions
`docker-compose.yml`	Déploiement local multi-conteneurs

3.3 API REST du backend

Table 3.2 – Endpoints de l'API Myway

Méthode	Route	Description
GET	`/api/modules`	Liste tous les modules
POST	`/api/modules`	Crée un module
PUT	`/api/modules/:id`	Met à jour un module
DELETE	`/api/modules/:id`	Supprime un module
GET	`/api/modules/:id/todos`	Todos d'un module
POST	`/api/modules/:id/todos`	Crée une todo
PATCH	`/api/todos/:id/toggle`	Bascule l'état d'une todo
POST	`/api/modules/:id/resources`	Ajoute une ressource
GET	`/api/uploads/:filename`	Télécharge un fichier
GET	`/api/health`	Contrôle de santé

3.4 Dockerisation de l'application

3.4.1 Dockerfile du backend

```
FROM node:18-alpine
WORKDIR /app
# Build tools required to compile sqlite3 native bindings
RUN apk add --no-cache python3 make g++
# Copy package files
COPY package*.json ./
# Install dependencies (allow scripts so sqlite3 native bindings are
    compiled)
RUN npm install --production
# Copy application code
COPY . .
# Create uploads directory
RUN mkdir -p uploads
# Expose port
EXPOSE 5000
# Start the server
CMD ["node", "server.js"]
```

Listing 3.1 – Dockerfile backend (Node.js/Express)

3.4.2 Dockerfile du frontend

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Listing 3.2 – Dockerfile frontend (React/Vite)

3.4.3 Docker Compose

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    volumes:
      - sqlite_data:/app/database
    environment:
      - PORT=5000
      - DB_PATH=/app/database/myway.db

  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    environment:
      - VITE_API_URL=http://backend:5000/api

volumes:
  sqlite_data:
```

Listing 3.3 – docker-compose.yml (déploiement local)

Chapitre 4

Infrastructure AWS avec Terraform et Ansible

4.1 Création du compte AWS et configuration IAM

4.1.1 Création du compte AWS Free Tier

La première étape du projet a consisté à créer un compte AWS en utilisant le niveau gratuit (*Free Tier*), qui offre 750 heures mensuelles d'instances `t3.micro` ainsi que $100 USD de crédits.

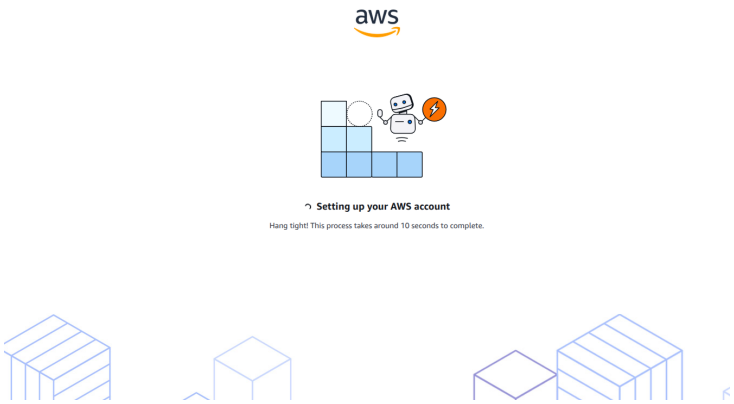

FIGURE 4.1 – Initialisation du compte AWS Free Tier

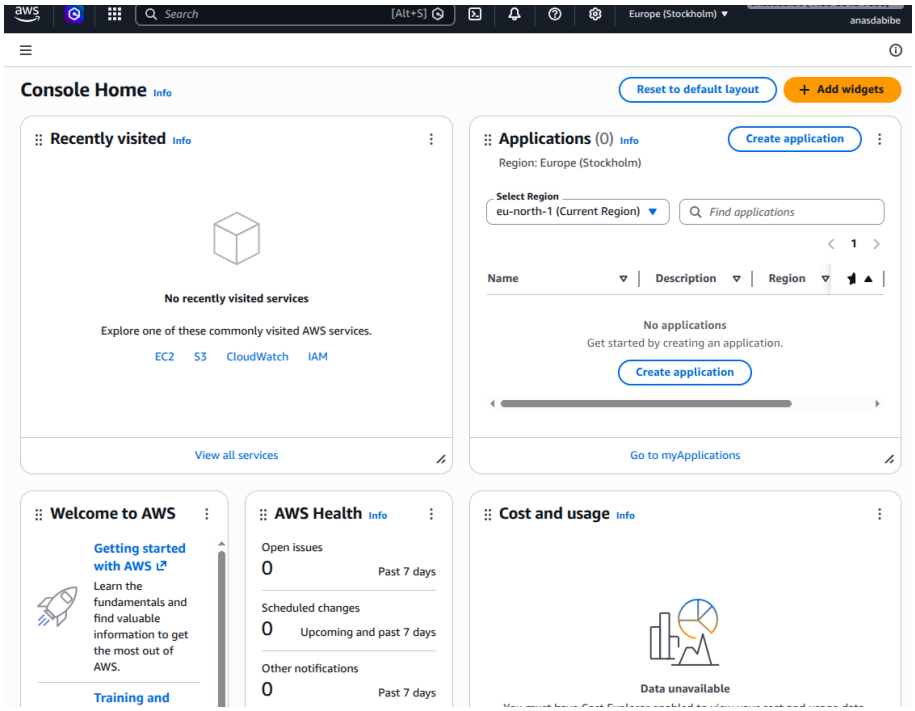

FIGURE 4.2 – Console AWS après création du compte

4.1.2 Configuration IAM

Un groupe IAM `project_devops_team` a été créé pour centraliser la gestion des permissions des membres de l'équipe. La politique `AmazonEC2ReadOnlyAccess` a été attachée au groupe.

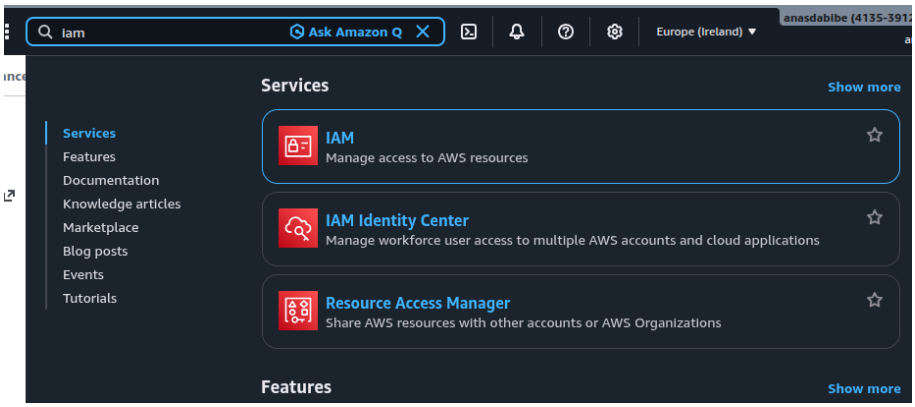

FIGURE 4.3 – Accès au service IAM depuis la console AWS

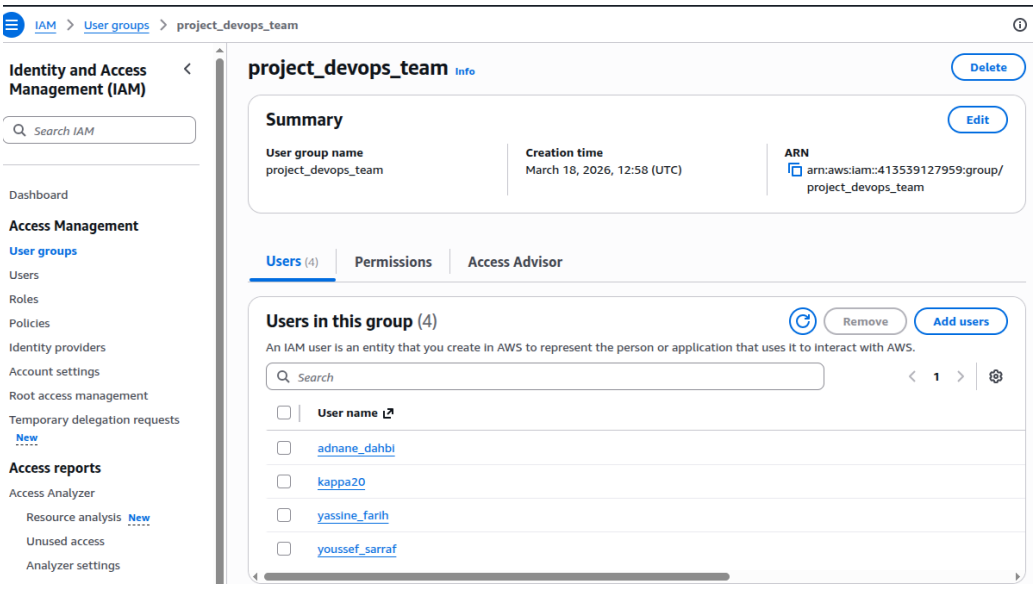

FIGURE 4.4 – Groupe IAM project_devops_team avec 4 membres

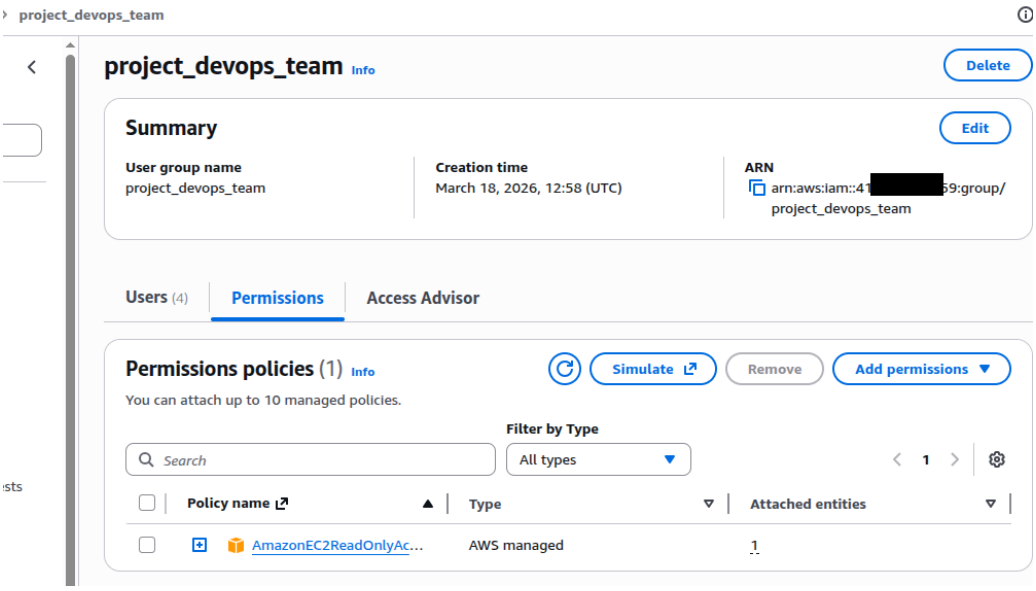

FIGURE 4.5 – Politique AmazonEC2ReadOnlyAccess attachée au groupe

TABLE 4.1 – Membres du groupe IAM project_devops_team

Nom d'utilisateur IAM	Rôle
Anass_dabibe	Membre
adnane_dahbi	Membre
yassine_farih	Membre
youssef_sarraf	Membre

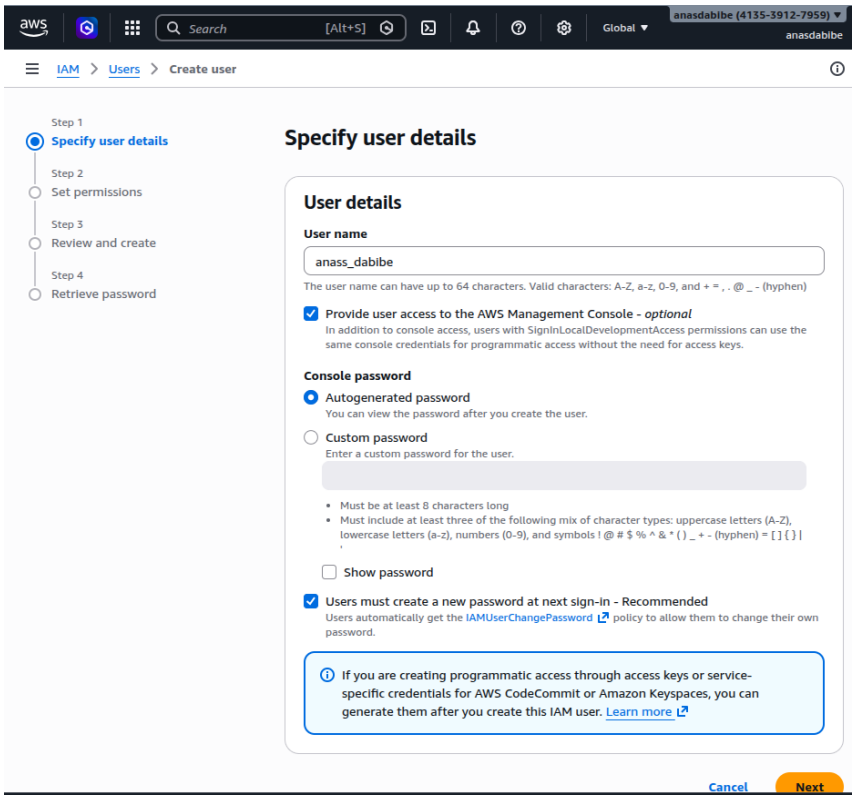

FIGURE 4.6 – Création d'un utilisateur IAM avec accès console

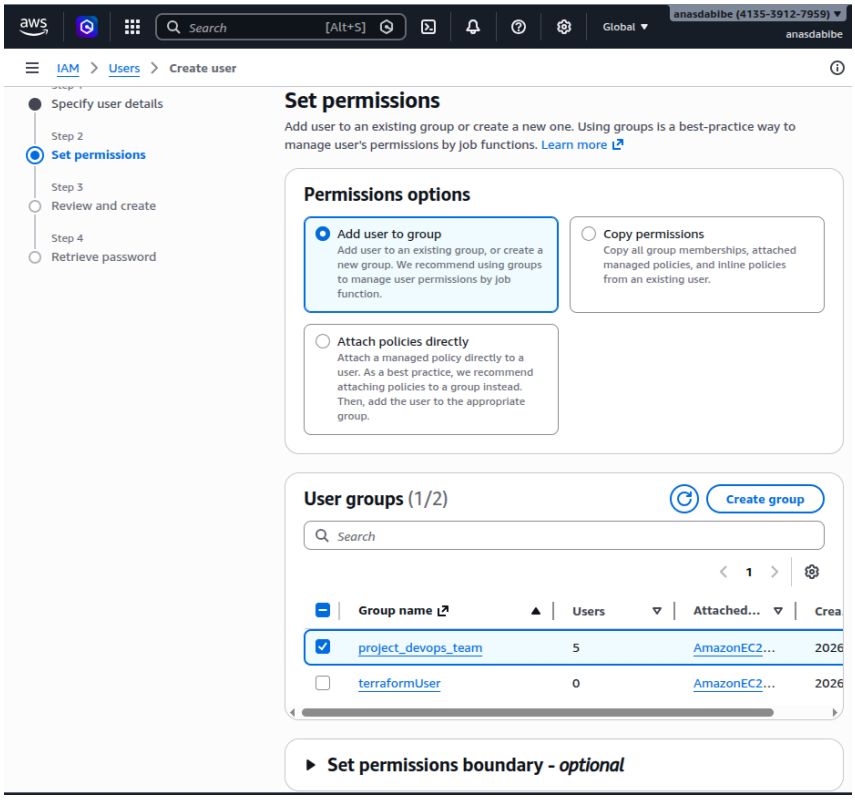

FIGURE 4.7 – Attachement de l'utilisateur au groupe project_devops_team

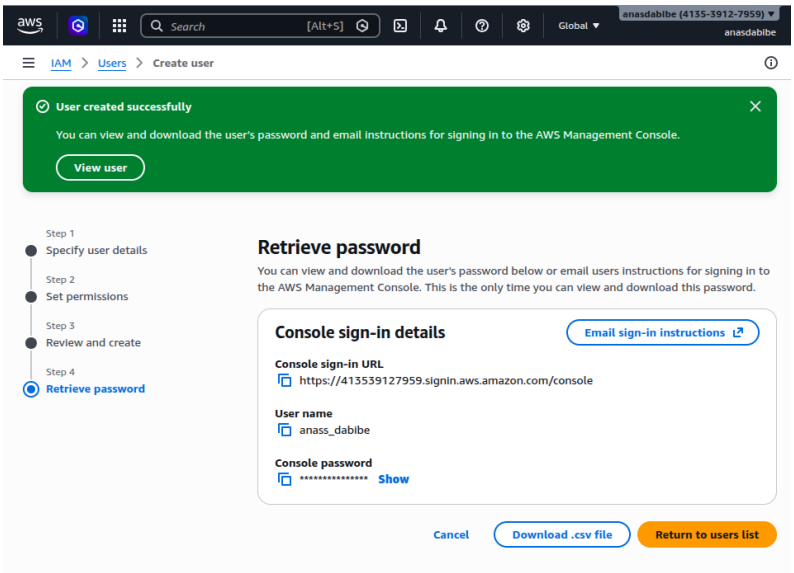

FIGURE 4.8 – Confirmation de création de l'utilisateur IAM

4.1.3 Génération des clés d'accès AWS

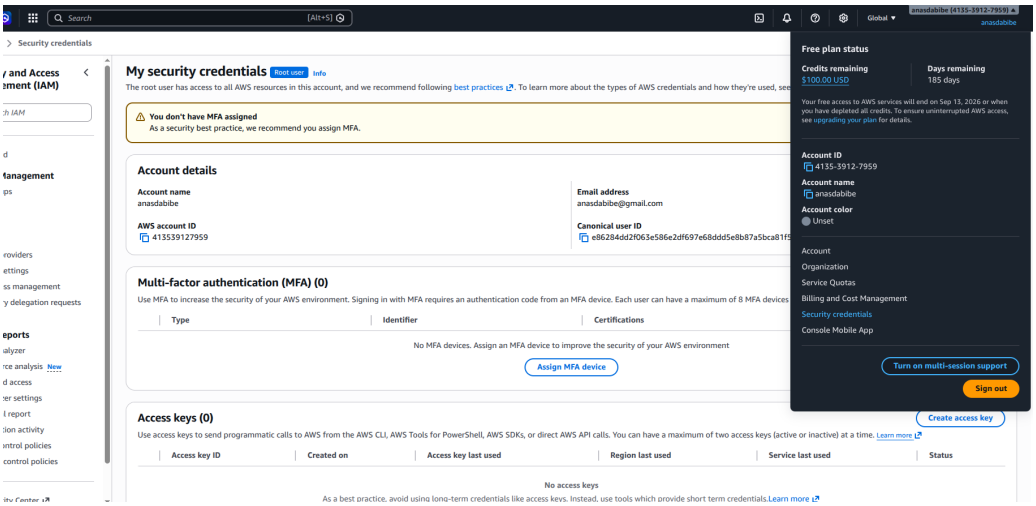

FIGURE 4.9 – Page de gestion des credentials AWS (Free Tier actif)

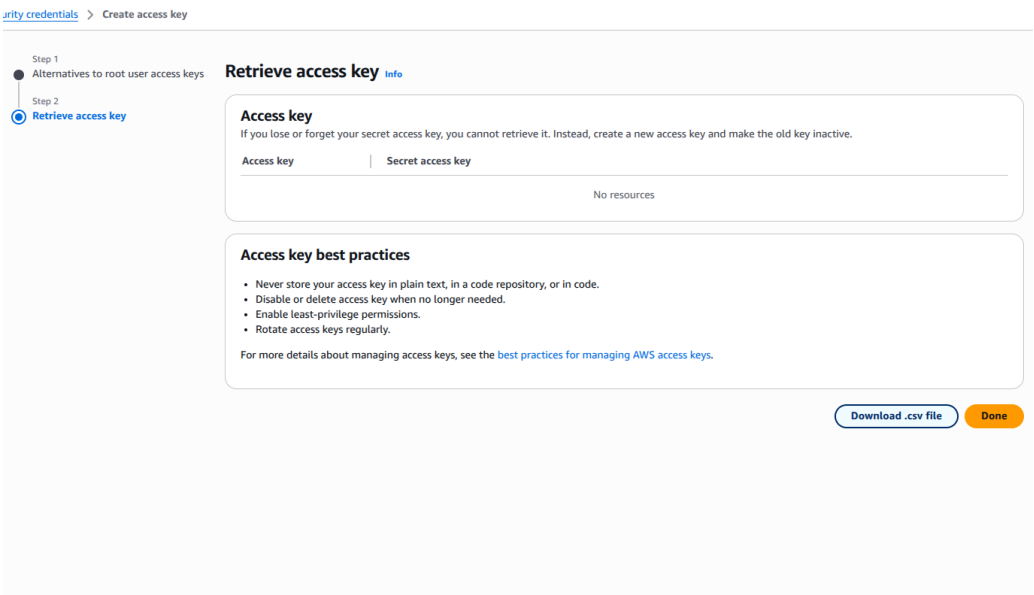

FIGURE 4.10 – Récupération de la clé d'accès AWS

4.1.4 Configuration de l'AWS CLI

```
aws configure set aws_access_key_id      <ACCESS_KEY_ID>
aws configure set aws_secret_access_key <SECRET_ACCESS_KEY>
aws configure set region                 eu-west-1
aws configure set output                 json

# Vérification de l'identité
aws sts get-caller-identity
```

Listing 4.1 – Configuration de l'AWS CLI

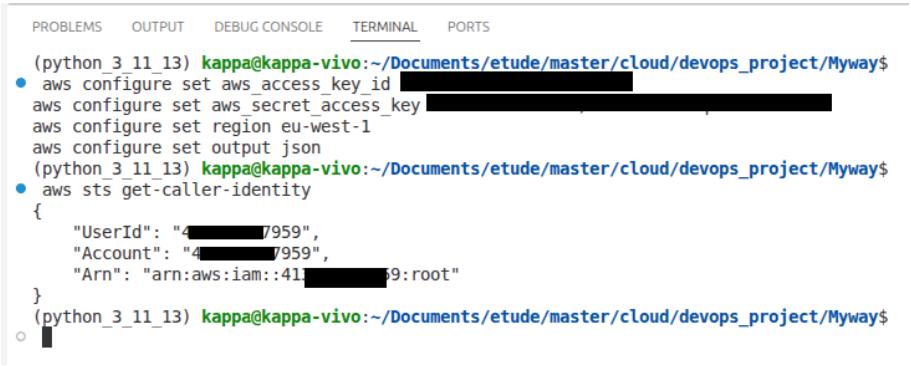

FIGURE 4.11 – Vérification de la configuration AWS CLI avec sts get-caller-identity

4.2 Infrastructure as Code avec Terraform

4.2.1 Vérification des prérequis

```
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project$
• terraform --version && ansible --version && aws --version && kubectl version --client
Terraform v1.14.7
on linux_amd64
ansible [core 2.19.7]
  config file = None
  configured module search path = ['/home/kappa/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /home/kappa/miniconda3/envs/python_3_11_13/lib/python3.11/site-packages/ansible
  ansible collection location = /home/kappa/.ansible/collections:/usr/share/ansible/collections
  executable location = /home/kappa/miniconda3/envs/python_3_11_13/bin/ansible
  python version = 3.11.13 (main, Jun  5 2025, 13:12:00) [GCC 11.2.0] (/home/kappa/miniconda3/envs/python_3_11_13/bin/python3.11)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
aws-cli/2.34.8 Python/3.13.11 Linux/6.17.0-14-generic exe/x86_64.ubuntu.24
Client Version: v1.35.1
Kustomize Version: v5.7.1
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project$
```

FIGURE 4.12 – Versions installées : Terraform v1.14.7, Ansible 2.19.7, AWS CLI 2.34.8, kubectl v1.35.1

4.2.2 Configuration Terraform

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region  # eu-west-1
}

resource "aws_vpc" "myway_vpc" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = "myway-vpc", Project = "myway" }
}

resource "aws_subnet" "myway_subnet" {
  vpc_id                  = aws_vpc.myway_vpc.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "eu-west-1a"
  map_public_ip_on_launch = true
  tags = { Name = "myway-subnet" }
}

resource "aws_internet_gateway" "myway_igw" {
```

```
  vpc_id = aws_vpc.myway_vpc.id
  tags   = { Name = "myway-igw" }
}

resource "aws_security_group" "myway_sg" {
  name   = "myway-sg"
  vpc_id = aws_vpc.myway_vpc.id
  ingress { from_port = 22;    to_port = 22;    protocol = "tcp";
     cidr_blocks = ["0.0.0.0/0"] }
  ingress { from_port = 6443;  to_port = 6443;  protocol = "tcp";
     cidr_blocks = ["0.0.0.0/0"] }
  ingress { from_port = 30000; to_port = 32767; protocol = "tcp";
     cidr_blocks = ["0.0.0.0/0"] }
  egress  { from_port = 0;     to_port = 0;     protocol = "-1";
     cidr_blocks = ["0.0.0.0/0"] }
}

resource "aws_instance" "k8s_master" {
  ami                    = data.aws_ami.amazon_linux_2.id
  instance_type          = "t3.micro"
  key_name               = aws_key_pair.myway_key.key_name
  subnet_id              = aws_subnet.myway_subnet.id
  vpc_security_group_ids = [aws_security_group.myway_sg.id]
  root_block_device { volume_size = 20 }
  tags = { Name = "k8s-master", Role = "master", Project = "myway" }
}

resource "aws_instance" "k8s_worker" {
  ami                    = data.aws_ami.amazon_linux_2.id
  instance_type          = "t3.micro"
  key_name               = aws_key_pair.myway_key.key_name
  subnet_id              = aws_subnet.myway_subnet.id
  vpc_security_group_ids = [aws_security_group.myway_sg.id]
  root_block_device { volume_size = 20 }
  tags = { Name = "k8s-worker", Role = "worker", Project = "myway" }
}
```

Listing 4.2 – terraform/main.tf – Infrastructure AWS

```
output "master_public_ip"   { value = aws_eip.master_eip.public_ip }
output "worker_public_ip"   { value = aws_eip.worker_eip.public_ip }
output "myway_frontend_url" { value = "http://${aws_eip.worker_eip.
    public_ip}:30080" }
output "myway_backend_url"  { value = "http://${aws_eip.worker_eip.
    public_ip}:30500" }
output "prometheus_url"     { value = "http://${aws_eip.master_eip.
    public_ip}:30090" }
output "grafana_url"        { value = "http://${aws_eip.master_eip.
    public_ip}:30030" }
```

Listing 4.3 – terraform/outputs.tf – Sorties Terraform

4.2.3 Déploiement Terraform

```
cd terraform/
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/terraform$
terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

FIGURE 4.13 – Initialisation de Terraform (`terraform init`)

```
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/terraform$
terraform plan
data.aws_ami.amazon_linux_2: Reading...
aws_vpc.myway_vpc: Refreshing state... [id=vpc-0d9e8041afc09c615]
aws_key_pair.myway_key: Refreshing state... [id=devops-myway-key]
data.aws_ami.amazon_linux_2: Read complete after 1s [id=ami-08ae1035a3a101a88]
aws_internet_gateway.myway_igw: Refreshing state... [id=igw-04706956aa9e8f140]
aws_subnet.myway_subnet: Refreshing state... [id=subnet-0d38adf293688b31b]
aws_security_group.myway_sg: Refreshing state... [id=sg-0387330721bdaeab0]
aws_route_table.myway_rt: Refreshing state... [id=rtb-086db0ebe19f752ff]
aws_route_table_association.myway_rta: Refreshing state... [id=rtbassoc-056899a5d6c58bfb5]
aws_instance.k8s_worker: Refreshing state... [id=i-0435992b869896d0c]
aws_instance.k8s_master: Refreshing state... [id=i-0d98a41f0bd49d062]

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may have affecte

  # aws_instance.k8s_master has changed
  ~ resource "aws_instance" "k8s_master" {
        id                                   = "i-0d98a41f0bd49d062"
      - public_ip                            = "108.129.230.81" -> null
        tags                                 = {
            "Name"    = "k8s-master"
            "Project" = "myway"
            "Role"    = "master"
        }
        # (38 unchanged attributes hidden)

        # (8 unchanged blocks hidden)
    }

  # aws_instance.k8s_worker has changed
  ~ resource "aws_instance" "k8s_worker" {
```

FIGURE 4.14 – Plan Terraform (`terraform plan`)

```
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/terraform$
 terraform output

grafana_url = "http://[redacted]222:30030"
master_instance_id = "i-0d98a41f0bd49d062"
master_private_ip = "10.0.1.106"
master_public_ip = "[redacted].81"
myway_backend_url = "http://[redacted].222:30500"
myway_frontend_url = "http:/[redacted].222:30080"
prometheus_url = "http:/[redacted]:30090"
ssh_command_master = "ssh -i ~/.ssh/id_rsa ec2-user@[redacted].81"
ssh_command_worker = "ssh -i ~/.ssh/id_rsa ec2-user@[redacted].222"
worker_instance_id = "i-0435992b869896d0c"
worker_private_ip = "10.0.1.172"
worker_public_ip = "[redacted].222"
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/terraform$
```

FIGURE 4.15 – Sorties Terraform après `terraform apply`

4.2.4 Adresses IP Élastiques

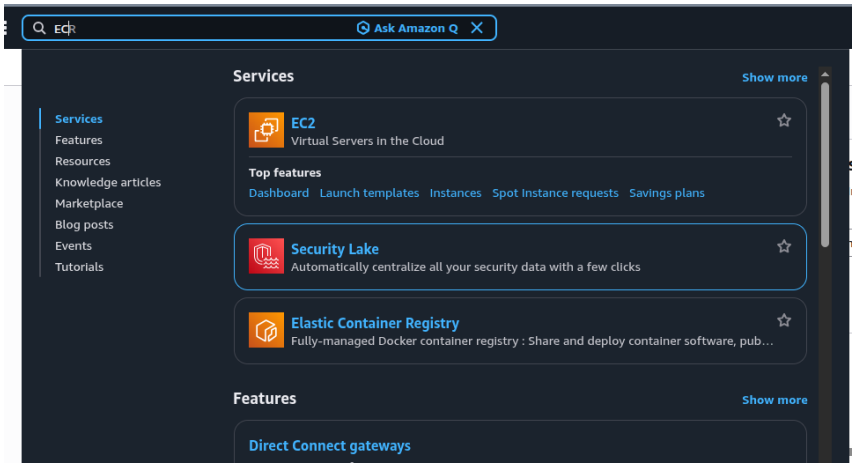

FIGURE 4.16 – Navigation vers le service EC2 dans la console AWS

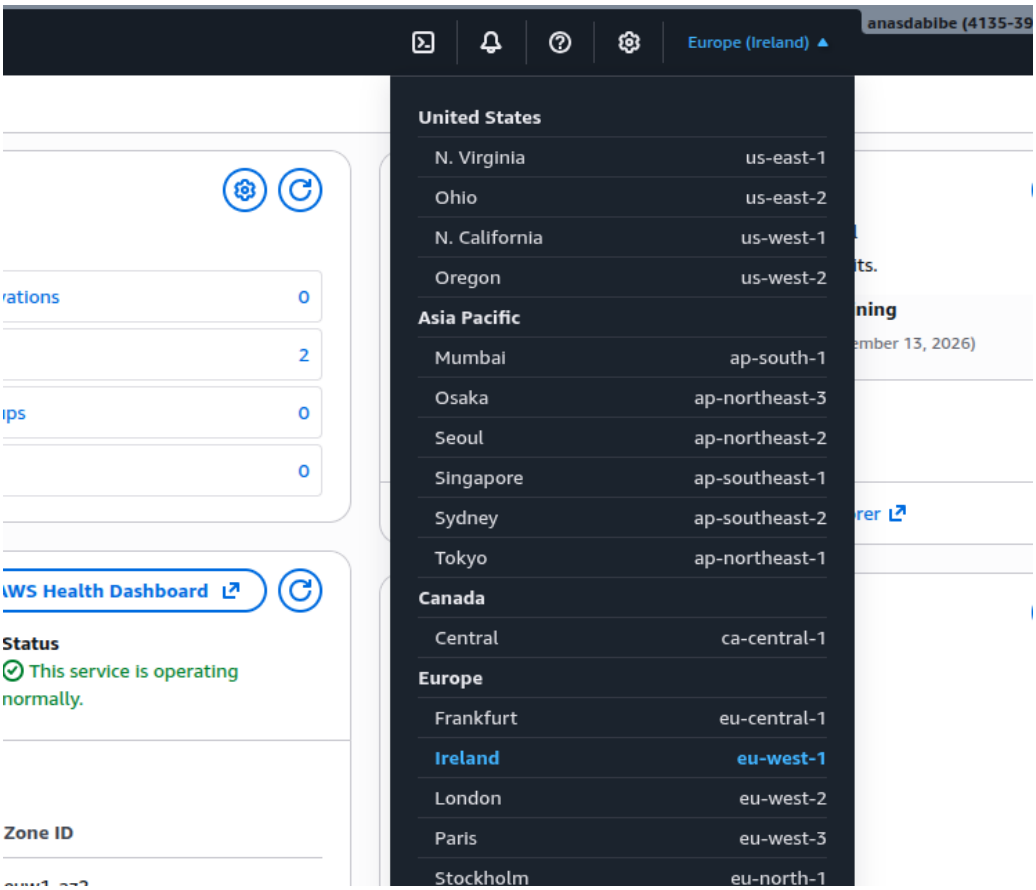

FIGURE 4.17 – Sélection de la région eu-west-1 (Irlande)

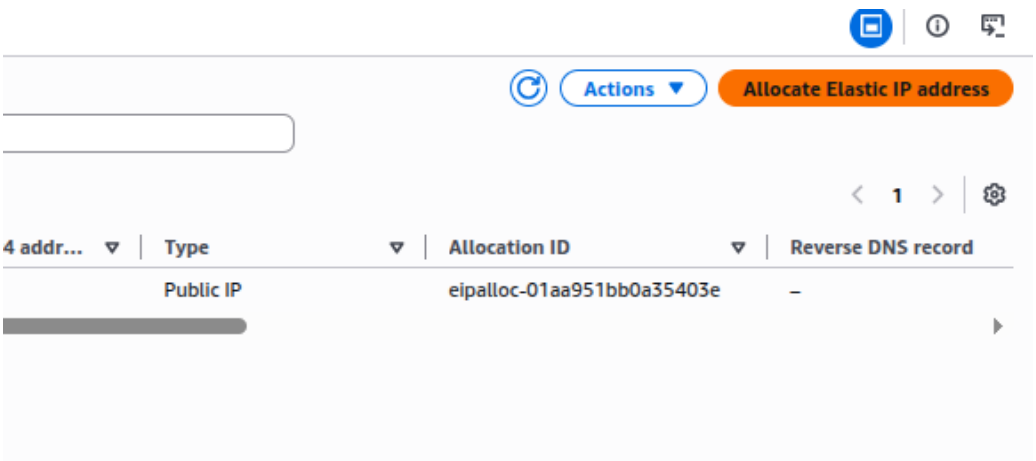

FIGURE 4.18 – Allocation d'une Elastic IP depuis EC2

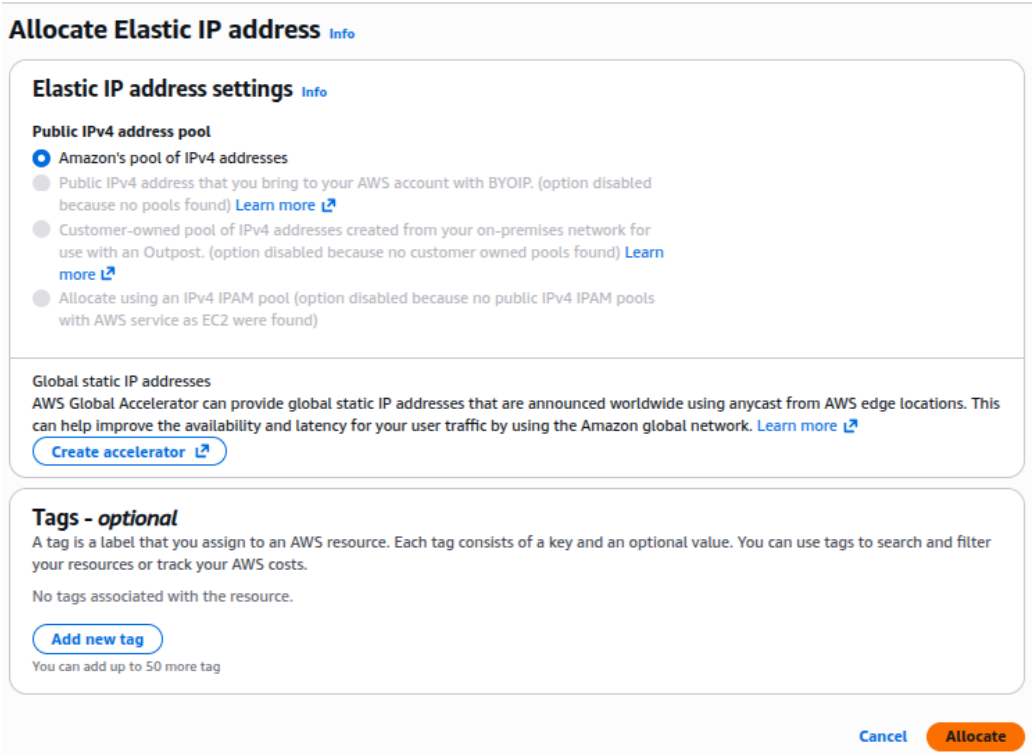

FIGURE 4.19 – Paramètres d'allocation de l'Elastic IP

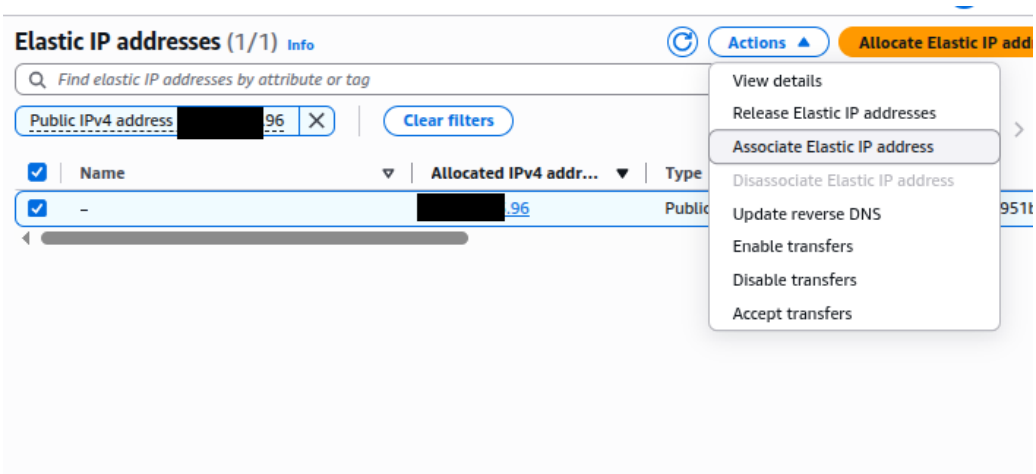

FIGURE 4.20 – Association de l'Elastic IP à l'instance k8s-master

4.2.5 Instances EC2 en production

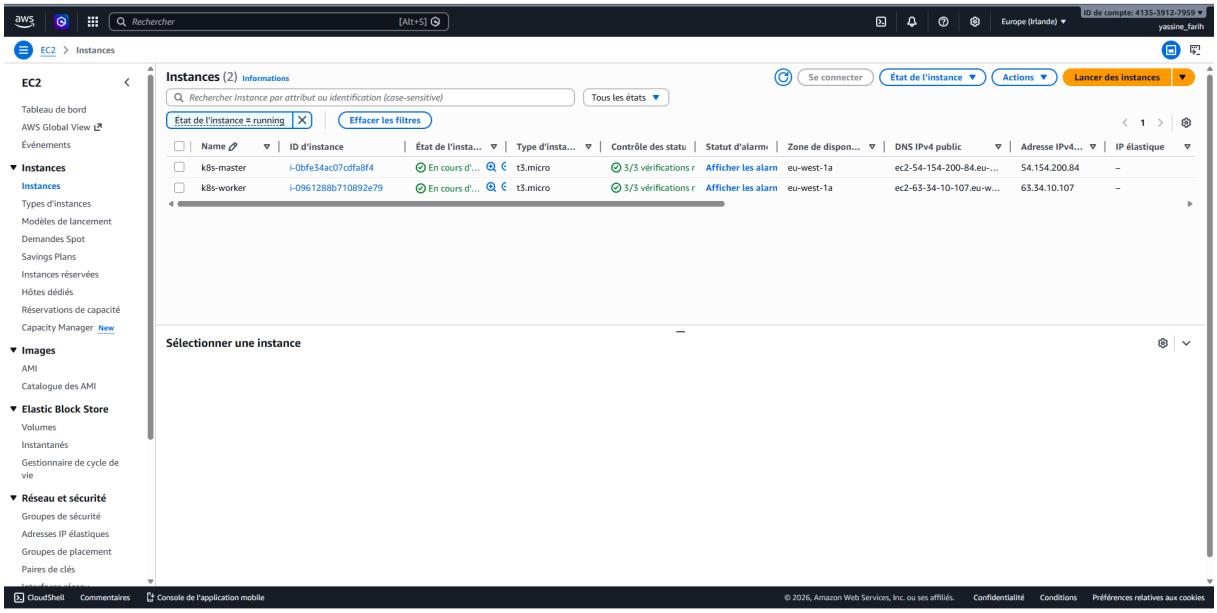

FIGURE 4.21 – Console AWS EC2 – instances k8s-master et k8s-worker en état Running

TABLE 4.2 – Instances EC2 déployées (dernier déploiement)

Nom	ID Instance	Type	IP Publique	Rôle
k8s-master	i-0bfe34ac07cdfa8f4	t3.micro	54.154.200.84	Control Plane
k8s-worker	i-0961288b710892e79	t3.micro	63.34.10.107	Worker Node

Note sur la rotation des adresses IP

Dans le cadre du Free Tier AWS, l'infrastructure a été provisionnée, testée, puis détruite (`terraform destroy`) à plusieurs reprises afin de maîtriser la consommation des 750h mensuelles d'instances `t3.micro`. Chaque nouveau `terraform apply` génère de nouvelles adresses IP publiques et privées. C'est pourquoi les captures d'écran issues de sessions différentes (Terraform output, console AWS, terminal SSH, Grafana) affichent des adresses IP distinctes – par exemple `18.200.244.174` dans le terminal SSH et `ip-10-0-1-171` dans Grafana correspondent à des sessions de déploiement antérieures. Les Elastic IPs ont partiellement résolu ce problème pour les déploiements finaux.

4.3 Configuration automatisée avec Ansible

4.3.1 Inventaire Ansible

```
[master]
k8s-master ansible_host=<MASTER_ELASTIC_IP> ansible_user=ec2-user

[workers]
k8s-worker ansible_host=<WORKER_ELASTIC_IP> ansible_user=ec2-user

[all:vars]
ansible_python_interpreter=auto
ansible_ssh_private_key_file=~/devops_project/ssh/myway-key
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
k8s_version=1.29.0
pod_network_cidr=10.244.0.0/16
```

Listing 4.4 – ansible/inventory.ini

```
ble > inventory.ini
# Ansible inventory for Myway DevOps project
# Replace <MASTER_PUBLIC_IP> and <WORKER_PUBLIC_IP> with actual IPs from 'terraform output'

[master]
k8s-master ansible_host=[redacted]96 ansible_user=ec2-user

[workers]
k8s-worker ansible_host=[redacted].147 ansible_user=ec2-user

# All hosts group
[all:vars]
ansible_python_interpreter=auto
ansible_ssh_private_key_file=/home/kappa/Documents/etude/master/cloud/devops_project/ssh/myway-key
ansible_ssh_common_args='-o StrictHostKeyChecking=no'

# Kubernetes specific variables
k8s_version=1.29.0
pod_network_cidr=10.244.0.0/16
```

FIGURE 4.22 – Fichier inventory.ini mis à jour avec les IPs élastiques

```
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project$
cd ansible/
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/ansible$
ansible all -m ping
k8s-master | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
k8s-worker | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

FIGURE 4.23 – Test de connectivité Ansible (`ansible all -m ping`) – SUCCESS sur les deux noeuds

4.3.2 Playbook 1 : Installation de Docker

```yaml
---
- name: Installer Docker sur tous les noeuds Kubernetes
  hosts: all
  become: true
  tasks:
    - name: Mettre a jour tous les paquets systeme
      yum: name=* state=latest
    - name: Installer Docker
      yum:
        name: [docker, docker-compose-plugin]
        state: present
    - name: Demarrer et activer le service Docker
      systemd: name=docker state=started enabled=true
    - name: Ajouter ec2-user au groupe docker
      user: name=ec2-user groups=docker append=true
```

Listing 4.5 – ansible/playbooks/01-install-docker.yml

```
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/ansible$
ansible-playbook playbooks/01-install-docker.yml

PLAY [Installer Docker sur tous les noeuds Kubernetes] ***********************************

TASK [Gathering Facts] ********************************************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Mettre à jour tous les paquets système] *********************************************
changed: [k8s-worker]
changed: [k8s-master]

TASK [Installer Docker] *******************************************************************
changed: [k8s-master]
changed: [k8s-worker]

TASK [Créer le groupe docker] *************************************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Démarrer et activer le service Docker] **********************************************
changed: [k8s-worker]
changed: [k8s-master]

TASK [Ajouter ec2-user au groupe docker] **************************************************
changed: [k8s-worker]
changed: [k8s-master]

TASK [Réinitialiser la connexion SSH (pour appliquer le groupe docker)] *******************

TASK [Réinitialiser la connexion SSH (pour appliquer le groupe docker)] *******************
```

FIGURE 4.24 – Exécution du playbook 01-install-docker.yml – Docker installé sur master et worker

4.3.3 Playbook 2 : Installation de Kubernetes

```yaml
---
- name: Installer les composants Kubernetes sur tous les noeuds
  hosts: all
  become: true
  tasks:
    - name: Desactiver le swap (permanent)
      replace:
        path: /etc/fstab
        regexp: '^([^#].*\sswap\s.*)$'
        replace: '# \1'
    - name: Ajouter le repository officiel Kubernetes
      yum_repository:
        name: kubernetes
        baseurl: https://pkgs.k8s.io/core:/stable:/v1.29/rpm/
        enabled: true
        gpgcheck: true
    - name: Installer kubeadm, kubelet et kubectl version 1.29.0
      yum:
        name: [kubeadm-1.29.0, kubelet-1.29.0, kubectl-1.29.0]
        state: present
    - name: Configurer les parametres sysctl pour Kubernetes
      sysctl:
        name: "{{ item.key }}"
        value: "{{ item.value }}"
        state: present
      loop:
        - { key: net.bridge.bridge-nf-call-iptables, value: "1" }
        - { key: net.ipv4.ip_forward, value: "1" }
```

Listing 4.6 – ansible/playbooks/02-install-kubernetes.yml (extrait)

```
 ansible-playbook playbooks/02-install-kubernetes.yml

PLAY [Installer les composants Kubernetes sur tous les noeuds] ****************************

TASK [Gathering Facts] *********************************************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Désactiver le swap dans /etc/fstab (permanent)] **************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Désactiver le swap immédiatement (sans redémarrer)] **********************************
skipping: [k8s-master]
skipping: [k8s-worker]

TASK [Ajouter la clé GPG de sécurité du repo Kubernetes] ***********************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Ajouter le repository officiel Kubernetes] *******************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Installer kubeadm, kubelet et kubectl version 1.29.0] ********************************
changed: [k8s-worker]
changed: [k8s-master]

TASK [Activer kubelet au démarrage du système] *********************************************
changed: [k8s-worker]
changed: [k8s-master]

TASK [Charger le module kernel br_netfilter] ***********************************************
ok: [k8s-master]
ok: [k8s-worker]

TASK [Charger le module kernel overlay] ****************************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Rendre les modules permanents au reboot] *********************************************
changed: [k8s-worker]
changed: [k8s-master]

TASK [Configurer les paramètres sysctl pour Kubernetes] ************************************
changed: [k8s-worker] => (item=net.bridge.bridge-nf-call-iptables)
changed: [k8s-master] => (item=net.bridge.bridge-nf-call-iptables)
changed: [k8s-worker] => (item=net.bridge.bridge-nf-call-ip6tables)
changed: [k8s-master] => (item=net.bridge.bridge-nf-call-ip6tables)
changed: [k8s-worker] => (item=net.ipv4.ip_forward)
changed: [k8s-master] => (item=net.ipv4.ip_forward)

TASK [Sauvegarder les paramètres sysctl (persistent après reboot)] *************************
changed: [k8s-worker]
changed: [k8s-master]

TASK [Vérifier que kubelet est installé] ***************************************************
ok: [k8s-worker]
ok: [k8s-master]

TASK [Afficher la version de kubelet installée] ********************************************
ok: [k8s-master] => {
    "msg": "✅ Kubernetes installé : Kubernetes v1.29.0"
}
ok: [k8s-worker] => {
    "msg": "✅ Kubernetes installé : Kubernetes v1.29.0"
}
```


FIGURE 4.25 – Exécution du playbook 02-install-kubernetes.yml – Kubernetes v1.29.0 installé

4.3.4 Playbook 3 : Initialisation du noeud maître

```yaml
---
- name: Initialiser le Master Node Kubernetes
  hosts: master
  become: true
  tasks:
    - name: Initialiser le cluster Kubernetes avec kubeadm
      command: >
        kubeadm init
        --pod-network-cidr={{ pod_network_cidr }}
        --apiserver-advertise-address={{ ansible_host }}
      when: not cluster_initialized.stat.exists
    - name: Copier admin.conf dans .kube/config
      copy:
        src: /etc/kubernetes/admin.conf
        dest: /home/ec2-user/.kube/config
        owner: ec2-user
        remote_src: true
    - name: Installer Flannel CNI
      become_user: ec2-user
      command: kubectl apply -f
        https://raw.githubusercontent.com/flannel-io/flannel/master/
            Documentation/kube-flannel.yml
    - name: Generer la commande join pour les workers
      command: kubeadm token create --print-join-command
      register: join_command
```

Listing 4.7 – ansible/playbooks/03-init-master.yml (extrait)

```
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/ansible$
 ansible-playbook playbooks/03-init-master.yml

PLAY [Initialiser le Master Node Kubernetes] ***********************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [k8s-master]

TASK [Initialiser le cluster Kubernetes avec kubeadm] **************************************************************
ok: [k8s-master]

TASK [Confirmer l'initialisation] **********************************************************************************
skipping: [k8s-master]

TASK [Créer le répertoire .kube pour ec2-user] *********************************************************************
ok: [k8s-master]

TASK [Copier admin.conf dans .kube/config] *************************************************************************
ok: [k8s-master]

TASK [Installer Flannel CNI (réseau des pods)] *********************************************************************
changed: [k8s-master]

TASK [Attendre que Flannel soit démarré] ***************************************************************************
changed: [k8s-master]

TASK [Générer la commande join pour les workers] *******************************************************************
changed: [k8s-master]

TASK [Sauvegarder la commande join sur la machine locale] **********************************************************
changed: [k8s-master -> localhost]

TASK [Afficher la confirmation] ************************************************************************************
ok: [k8s-master] => {
    "msg": "✅ Commande join sauvegardée dans /tmp/k8s_join_command.sh"
}

TASK [Attendre que le master soit prêt] ****************************************************************************
changed: [k8s-master]

TASK [Afficher les noeuds du cluster] ******************************************************************************
ok: [k8s-master]

TASK [Résultat — état des noeuds] **********************************************************************************
ok: [k8s-master] => {
    "msg": "NAME                                      STATUS   ROLES           AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   O
>       Amazon Linux 2   4.14.355-280.714.amzn2.x86_64   containerd://2.1.5"
}

TASK [Afficher les infos du cluster] *******************************************************************************
ok: [k8s-master]

TASK [Résultat — infos du cluster] *********************************************************************************
ok: [k8s-master] => {
    "msg": "\u001b[0;32mKubernetes control plane\u001b[0m is running at \u001b[0;33mhttps://10.0.1.52:6443\u001b[0m\n\u001b[0;
uster problems, use 'kubectl cluster-info dump'."
}

PLAY RECAP *********************************************************************************************************
k8s-master                 : ok=14   changed=5    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/ansible$
```

FIGURE 4.26 – Exécution du playbook 03-init-master.yml – Flannel installé, commande join générée

4.3.5 Playbook 4 : Jonction du worker au cluster

```yaml
---
- name: Joindre le Worker au Cluster Kubernetes
  hosts: workers
  become: true
  tasks:
    - name: Executer la commande join
      command: bash /tmp/k8s_join_command.sh
      when: not already_joined.stat.exists
    - name: Attendre que le worker soit en etat Ready
      delegate_to: "{{ groups['master'][0] }}"
      become_user: ec2-user
      command: kubectl wait --for=condition=Ready node/k8s-worker --
          timeout=120s
```

Listing 4.8 – ansible/playbooks/04-join-worker.yml (extrait)

```
(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/ansible$
ansible-playbook playbooks/04-join-worker.yml

PLAY [Joindre le Worker au Cluster Kubernetes] ***********************************************************************************************************************

TASK [Gathering Facts] ***********************************************************************************************************************************************
ok: [k8s-worker]

TASK [Copier la commande join vers le worker] ************************************************************************************************************************
ok: [k8s-worker]

TASK [Afficher la commande join (pour vérification)] *****************************************************************************************************************
ok: [k8s-worker]

TASK [Contenu de la commande join] ***********************************************************************************************************************************
ok: [k8s-worker] => {
    "msg": "Commande join : kubeadm join 10.0.1.52:6443 --token 6l2hsa.vdbfrrh1qq4wb21k --discovery-token-ca-cert-hash sha256:0500a34d2264af0611b5c9ccbacedfedae04ecf1f0ba4de595f
}

TASK [Exécuter la commande join pour rejoindre le cluster] ***********************************************************************************************************
ok: [k8s-worker]

TASK [Confirmer que le worker a rejoint le cluster] ******************************************************************************************************************
skipping: [k8s-worker]

TASK [Attendre que le worker soit en état Ready] *********************************************************************************************************************
changed: [k8s-worker -> k8s-master(18.200.244.174)]

TASK [Afficher l'état final de tous les noeuds] ***********************************************************************************************************************
ok: [k8s-worker -> k8s-master(18.200.244.174)]

TASK [Résultat final — cluster complet] ******************************************************************************************************************************
ok: [k8s-worker] => {
    "msg": "NAME                                STATUS   ROLES           AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE        KERNEL-VERSION                 CO
ne>       Amazon Linux 2   4.14.355-280.714.amzn2.x86_64   containerd://2.1.5\nip-10-0-1-52.eu-west-1.compute.internal    Ready    control-plane   6m27s   v1.29.0   10.0.1.52
}

PLAY RECAP ***********************************************************************************************************************************************************
k8s-worker                 : ok=8    changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project/ansible$
```

FIGURE 4.27 – Worker joint au cluster avec succès – état Ready en 6m27s

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(python_3_11_13) kappa@kappa-vivo:~/Documents/etude/master/cloud/devops_project$
 ssh -i ssh/myway-key ec2-user@18.200.244.174
Last login: Mon Mar 16 13:23:22 2026 from 105.68.180.189
       __|  __|_  )
       _|  (     /   Amazon Linux 2
      ___|\___|___|
      https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-1-52 ~]$ kubectl get nodes
NAME                                       STATUS   ROLES           AGE    VERSION
ip-10-0-1-137.eu-west-1.compute.internal   Ready    <none>          3m38s  v1.29.0
ip-10-0-1-52.eu-west-1.compute.internal    Ready    control-plane   8m19s  v1.29.0
[ec2-user@ip-10-0-1-52 ~]$
```

FIGURE 4.28 – Vérification finale – les deux noeuds sont Ready (kubectl get nodes)

4.3.6 Playbook 5 : Déploiement du Monitoring

Ce playbook automatise le déploiement de la stack Prometheus et Grafana sur le cluster. Il évite d'avoir à appliquer manuellement les manifests Kubernetes.

```yaml
- name: Installer la stack de monitoring
  hosts: master
  become: true
  tasks:
    - name: Appliquer les manifests Prometheus et Grafana
      become_user: ec2-user
      command: >
        kubectl apply -f /tmp/k8s-myway/monitoring/prometheus.yml
        -f /tmp/k8s-myway/monitoring/node-exporter.yml
        -f /tmp/k8s-myway/monitoring/grafana.yml
```

Listing 4.9 – ansible/playbooks/05-install-monitoring.yml (extrait)

4.3.7 Playbook 6 : Installation du Dashboard Kubernetes

Pour faciliter la visualisation de l'état du cluster, ce dernier playbook installe le Kubernetes Dashboard officiel.

```yaml
- name: Installer Kubernetes Dashboard
  hosts: master
  become: true
  tasks:
    - name: Appliquer le manifest du Dashboard
      become_user: ec2-user
      command: kubectl apply -f /tmp/k8s-myway/dashboard.yml
```

Listing 4.10 – ansible/playbooks/06-install-dashboard.yml (extrait)

Chapitre 5

Déploiement Kubernetes

5.1 Manifests Kubernetes

5.1.1 Namespace

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: myway
  labels:
    app: myway
    environment: production
```

Listing 5.1 – k8s/namespace.yml

5.1.2 Déploiement du Backend

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myway-backend
  namespace: myway
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myway-backend
  template:
    metadata:
      labels:
        app: myway-backend
    spec:
      containers:
        - name: backend
```

```yaml
          image: kappa20/myway-backend:latest
          ports:
            - containerPort: 5000
          env:
            - name: PORT
              value: "5000"
            - name: DB_PATH
              value: "/app/data/myway.db"
          volumeMounts:
            - name: sqlite-data
              mountPath: /app/data
      volumes:
        - name: sqlite-data
          emptyDir: {}
          # Choix volontaire pour ce projet : SQLite est léger et suffisant
          # pour un environnement de démonstration. La limite de persistance
          # (données perdues au redémarrage du pod) est documentée en conclusion.
---
apiVersion: v1
kind: Service
metadata:
  name: myway-backend
  namespace: myway
spec:
  selector:
    app: myway-backend
  type: NodePort
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30500
```

Listing 5.2 – k8s/backend-deployment.yml

5.1.3 Déploiement du Frontend

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myway-frontend
  namespace: myway
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myway-frontend
  template:
    metadata:
      labels:
        app: myway-frontend
    spec:
      containers:
        - name: frontend
          image: kappa20/myway-frontend:latest
          ports:
            - containerPort: 80
          env:
            - name: VITE_API_URL
              value: "http://63.34.10.107:30500/api"
              # Note : cette valeur est injectée dynamiquement par le pipeline CI/CD
              # via un sed avant kubectl apply (voir job "deploy", étape
              # "Inject Worker IP in Frontend Manifest")
---
apiVersion: v1
kind: Service
metadata:
  name: myway-frontend
  namespace: myway
spec:
  selector:
    app: myway-frontend
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Listing 5.3 – k8s/frontend-deployment.yml

5.1.4 Monitoring Kubernetes

```yaml
#
# Prometheus     Configuration améliorée  [AMÉLIORÉ]
# Fichier : k8s/monitoring/prometheus.yml
#
# Améliorations par rapport à la version initiale :
#       ConfigMap avec prometheus.yml pour scraper les pods Myway
#       Volume monté pour persister la config
#       Règles d'alerte pour CPU/mémoire/pods down
#

#        ConfigMap : configuration de scraping

apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
  namespace: myway
  labels:
    app: prometheus
    component: monitoring
    project: myway
data:
  prometheus.yml: |
    global:
      scrape_interval:     15s   # fréquence de scraping
      evaluation_interval: 15s   # fréquence d'évaluation des règles

    rule_files:
      - "/etc/prometheus/rules/*.yml"

    scrape_configs:
      #        Prometheus lui-même

      - job_name: 'prometheus'
        static_configs:
          - targets: ['localhost:9090']

      #        Backend Myway
```

```yaml
    # Scrape automatique via annotations Kubernetes sur les pods
    - job_name: 'myway-backend'
      kubernetes_sd_configs:
        - role: pod
          namespaces:
            names: ['myway']
      relabel_configs:
        - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
          action: keep
          regex: "true"
        - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
          action: replace
          target_label: __metrics_path__
          regex: (.+)
        - source_labels: [__address__, __meta_kubernetes_pod_annotation_prometheus_io_port]
          action: replace
          regex: ([^:]+)(?::\d+)?;(\d+)
          replacement: $1:$2
          target_label: __address__
        - source_labels: [__meta_kubernetes_pod_label_app]
          target_label: app
        - source_labels: [__meta_kubernetes_namespace]
          target_label: namespace

    #        Node exporter (métriques système des n uds)

    - job_name: 'node-exporter'
      kubernetes_sd_configs:
        - role: pod
          namespaces:
            names: ['myway']
      relabel_configs:
        - source_labels: [__meta_kubernetes_pod_label_app]
          action: keep
          regex: node-exporter
        - source_labels: [__meta_kubernetes_pod_node_name]
          target_label: node
        - source_labels: [__address__]
          action: replace
          regex: ([^:]+)(?::\d+)?
          replacement: $1:9100
          target_label: __address__

```

```yaml
  #         Règles d'alerte


  alert-rules.yml: |
    groups:
      - name: myway-alerts
        rules:
          # Alerte si un pod est down depuis plus de 1 min
          - alert: PodDown
            expr: kube_pod_status_ready{namespace="myway", condition="true"} == 0
            for: 1m
            labels:
              severity: critical
            annotations:
              summary: "Pod {{ $labels.pod }} is down"
              description: "Pod {{ $labels.pod }} in namespace {{ $labels.namespace }} is not ready for more than 1 minute."

          # Alerte si CPU > 80% depuis 5 min
          - alert: HighCPUUsage
            expr: |
              sum(rate(container_cpu_usage_seconds_total{namespace="myway"}[5m])) by (pod)
              / sum(kube_pod_container_resource_limits{namespace="myway", resource="cpu"}) by (pod)
              > 0.8
            for: 5m
            labels:
              severity: warning
            annotations:
              summary: "High CPU usage on pod {{ $labels.pod }}"
              description: "CPU usage is above 80% for 5 minutes on pod {{ $labels.pod }}."

          # Alerte si mémoire > 85% depuis 5 min
          - alert: HighMemoryUsage
            expr: |
              sum(container_memory_working_set_bytes{namespace="myway"}) by (pod)
              / sum(kube_pod_container_resource_limits{namespace="myway", resource="memory"}) by (pod)
              > 0.85
            for: 5m
            labels:
              severity: warning
```

```yaml
            annotations:
              summary: "High memory usage on pod {{ $labels.pod }}"
              description: "Memory usage is above 85% for 5 minutes on pod {{ $labels.pod }}."

          # Alerte si le déploiement a 0 replica disponible
          - alert: DeploymentUnavailable
            expr: kube_deployment_status_available_replicas{namespace="myway"} == 0
            for: 2m
            labels:
              severity: critical
            annotations:
              summary: "Deployment {{ $labels.deployment }} has no available replicas"

---
#        Deployment Prometheus


apiVersion: apps/v1
kind: Deployment
metadata:
  name: prometheus
  namespace: myway
  labels:
    app: prometheus
    component: monitoring
    project: myway
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus
  template:
    metadata:
      labels:
        app: prometheus
        component: monitoring
    spec:
      # RBAC : Prometheus a besoin de lire les pods/nodes
      serviceAccountName: prometheus-sa
      containers:
        - name: prometheus
          image: prom/prometheus:v2.51.0    # version fixée (pas :latest en prod)
          args:
```

```yaml
            - "--config.file=/etc/prometheus/prometheus.yml"
            - "--storage.tsdb.path=/prometheus"
            - "--storage.tsdb.retention.time=7d"   # garder 7 jours de m
              étriques
            - "--web.console.libraries=/usr/share/prometheus/
              console_libraries"
            - "--web.console.templates=/usr/share/prometheus/consoles"
            - "--web.enable-lifecycle"             # permet le reload à
              chaud
          ports:
            - containerPort: 9090
              name: web
          volumeMounts:
            - name: prometheus-config
              mountPath: /etc/prometheus/prometheus.yml
              subPath: prometheus.yml
            - name: prometheus-config
              mountPath: /etc/prometheus/rules/alert-rules.yml
              subPath: alert-rules.yml
            - name: prometheus-data
              mountPath: /prometheus
          resources:
            requests:
              cpu: 100m
              memory: 256Mi
            limits:
              cpu: 500m
              memory: 512Mi
          livenessProbe:
            httpGet:
              path: /-/healthy
              port: 9090
            initialDelaySeconds: 30
            periodSeconds: 15
          readinessProbe:
            httpGet:
              path: /-/ready
              port: 9090
            initialDelaySeconds: 10
            periodSeconds: 10
      volumes:
        - name: prometheus-config
          configMap:
            name: prometheus-config
        - name: prometheus-data
          emptyDir: {}

```

```yaml
---
#        Service Prometheus


apiVersion: v1
kind: Service
metadata:
  name: prometheus
  namespace: myway
  labels:
    app: prometheus
spec:
  type: NodePort
  selector:
    app: prometheus
  ports:
    - name: web
      port: 9090
      targetPort: 9090
      nodePort: 30090

---
#        ServiceAccount + ClusterRole pour Prometheus

# Prometheus a besoin de lire les pods, nodes et services K8s
apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus-sa
  namespace: myway

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus-clusterrole
rules:
  - apiGroups: [""]
    resources: ["nodes", "pods", "services", "endpoints"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["extensions", "apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch"]
  - nonResourceURLs: ["/metrics"]
    verbs: ["get"]

---
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus-clusterrolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus-clusterrole
subjects:
  - kind: ServiceAccount
    name: prometheus-sa
    namespace: myway
```

Listing 5.4 – k8s/monitoring/prometheus.yml (extrait)

5.1.5 Horizontal Pod Autoscaling (HPA)

Pour garantir la haute disponibilité et l'optimisation des ressources, nous avons mis en place des politiques d'auto-scaling horizontal. Le HPA ajuste dynamiquement le nombre de réplicas des pods en fonction de l'utilisation du CPU.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myway-backend-hpa
  namespace: myway
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myway-backend
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Listing 5.5 – k8s/hpa/backend-hpa.yml

Le backend peut ainsi passer de 1 à 5 instances si la charge CPU moyenne dépasse 70%. Une configuration similaire a été appliquée au frontend avec un seuil de 80% et un maximum de 3 réplicas.

5.2 État du cluster Kubernetes

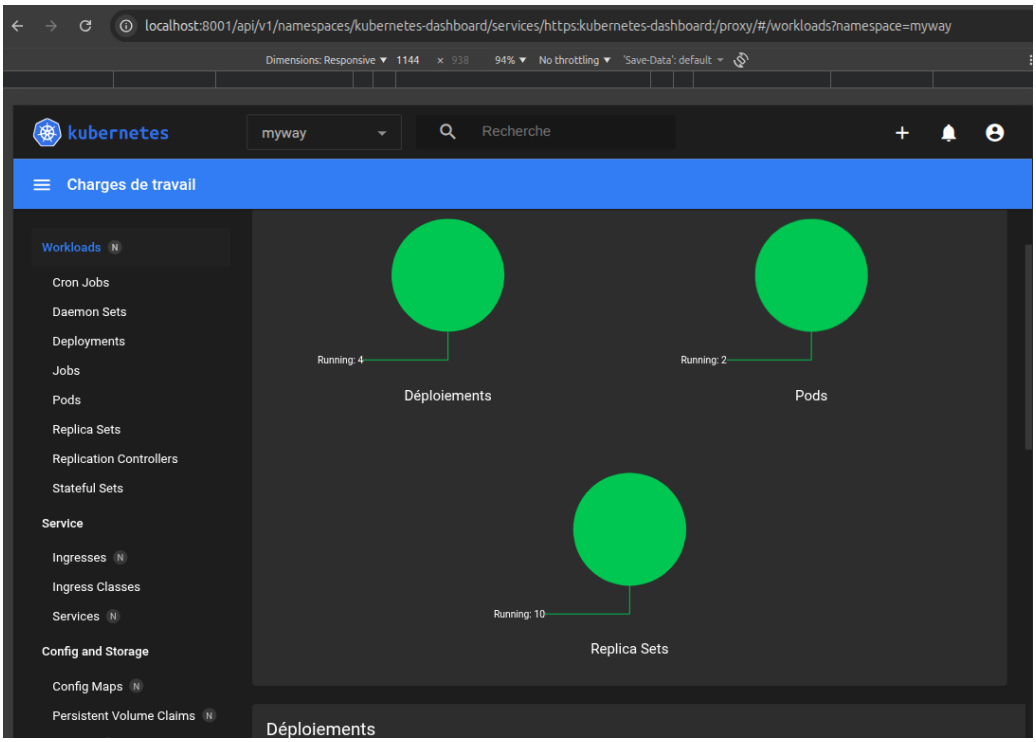

FIGURE 5.1 – Tableau de bord Kubernetes – namespace myway : 4 déploiements, 2 pods, 10 replica sets

TABLE 5.1 – Points d'accès aux services Myway

Service	NodePort	URL d'accès
Frontend	30080	`http://<WORKER_IP>:30080`
Backend API	30500	`http://<WORKER_IP>:30500/api`
Prometheus	30090	`http://<MASTER_IP>:30090`
Grafana	30030	`http://<MASTER_IP>:30030`

5.3 Aperçu de l'application déployée

Une fois le déploiement Kubernetes finalisé et les services exposés via NodePort, l'application Myway est pleinement opérationnelle. La figure suivante présente l'interface finale de la plateforme telle qu'elle apparaît pour l'utilisateur final, confirmant le bon fonctionnement de la communication entre le frontend React et l'API backend Node.js au sein du cluster AWS.

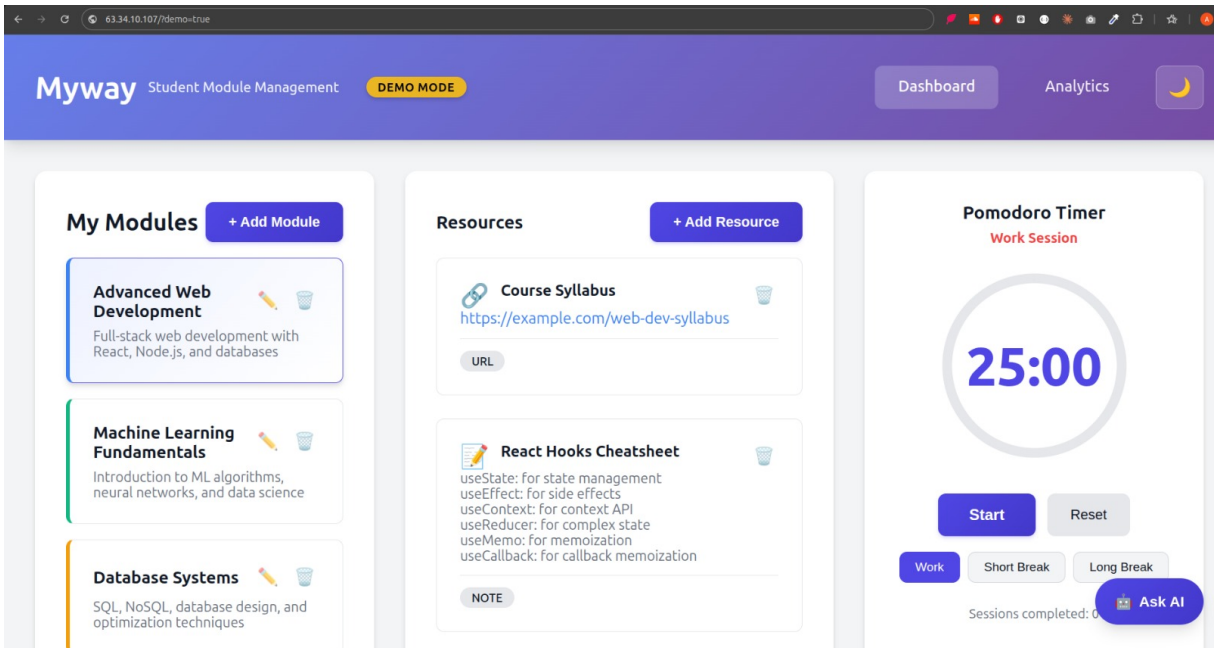

FIGURE 5.2 – Interface de l'application Myway en production sur le cluster Kubernetes

Chapitre 6

Pipeline CI/CD avec GitHub Actions

6.1 Vue d'ensemble du pipeline

Le pipeline CI/CD a été implémenté avec GitHub Actions. Il se déclenche automatiquement à chaque `git push` sur la branche `main` et enchaîne six jobs couvrant les aspects qualité, sécurité et déploiement.

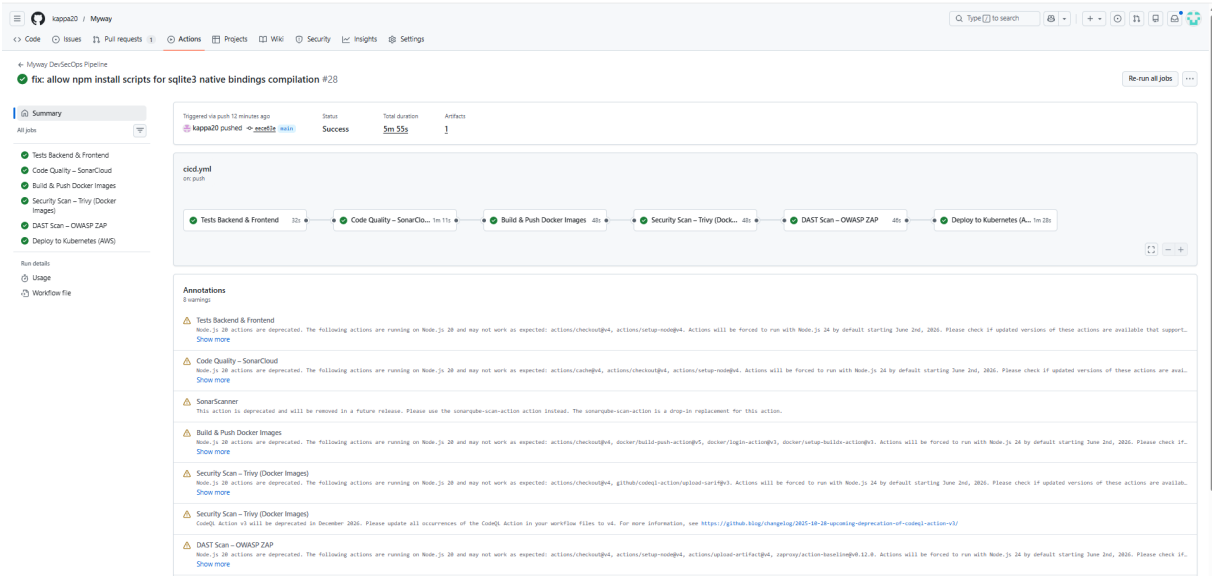

FIGURE 6.1 – Vue globale du pipeline DevSecOps – 6 jobs réussis, durée totale : 5m 55s

6.2 Configuration du pipeline

```
name: Myway DevSecOps Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

```yaml

jobs:

  # -- Job 1 : Tests ------------------------------------------
  test:
    name: Tests Backend & Frontend
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js (backend)
        uses: actions/setup-node@v4
        with: { node-version: '18', cache: 'npm',
                cache-dependency-path: backend/package-lock.json }
      - name: Install backend dependencies
        run: cd backend && npm ci
      - name: Run backend tests
        run: cd backend && npm test
      - name: Setup Node.js (frontend)
        uses: actions/setup-node@v4
        with: { node-version: '18', cache: 'npm',
                cache-dependency-path: frontend/package-lock.json }
      - name: Install frontend dependencies
        run: cd frontend && npm ci
      - name: Build frontend
        run: cd frontend && npm run build

  # -- Job 2 : Qualité du code --------------------------------
  code-quality:
    name: Code Quality - SonarCloud
    runs-on: ubuntu-latest
    needs: test
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - name: SonarCloud Scan
        uses: SonarSource/sonarcloud-github-action@master
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}

  # -- Job 3 : Build et push Docker ---------------------------
  build:
    name: Build & Push Docker Images
    runs-on: ubuntu-latest
    needs: code-quality
    steps:
      - uses: actions/checkout@v4
      - name: Set up Docker Buildx
```

```yaml
        uses: docker/setup-buildx-action@v3
      - name: Login to DockerHub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Build and push backend image
        uses: docker/build-push-action@v5
        with: { context: ./backend, push: true,
                tags: kappa20/myway-backend:latest }
      - name: Build and push frontend image
        uses: docker/build-push-action@v5
        with: { context: ./frontend, push: true,
                tags: kappa20/myway-frontend:latest }

  # -- Job 4 : Scan de securite Trivy -------------------------
  security-scan:
    name: Security Scan - Trivy
    runs-on: ubuntu-latest
    needs: build
    steps:
      - uses: actions/checkout@v4
      - name: Install Trivy
        run: sudo apt-get install -y trivy
      - name: Scan backend image - CRITICAL/HIGH
        run: trivy image --exit-code 0 --severity CRITICAL,HIGH
               kappa20/myway-backend:latest
      - name: Scan backend image - SARIF report
        run: trivy image --format sarif --output backend-trivy.sarif
               kappa20/myway-backend:latest
      - name: Upload backend SARIF to GitHub Security
        uses: github/codeql-action/upload-sarif@v3
        with: { sarif_file: backend-trivy.sarif }
      - name: Scan frontend image - CRITICAL/HIGH
        run: trivy image --exit-code 0 --severity CRITICAL,HIGH
               kappa20/myway-frontend:latest
      - name: Scan filesystem - dependances npm
        run: trivy fs --exit-code 0 backend/

  # -- Job 5 : DAST OWASP ZAP ---------------------------------
  dast-scan:
    name: DAST Scan - OWASP ZAP
    runs-on: ubuntu-latest
    needs: build
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
```

```yaml
        uses: actions/setup-node@v4
        with: { node-version: '18' }
      - name: Install backend dependencies
        run: cd backend && npm ci
      - name: Start backend server (mode test)
        run: |
          cd backend && node server.js &
          sleep 10
          curl -f http://localhost:5000/api/health
      - name: OWASP ZAP Baseline Scan
        uses: zaproxy/action-baseline@v0.12.0
        with: { target: 'http://localhost:5000/api', fail_action: false
            }
      - name: Upload ZAP HTML report
        uses: actions/upload-artifact@v4
        with: { name: zap-report, path: report_html.html }

  # -- Job 6 : Deploiement Kubernetes -------------------------
  deploy:
    name: Deploy to Kubernetes (AWS)
    runs-on: ubuntu-latest
    needs: [security-scan, dast-scan]
    steps:
      - uses: actions/checkout@v4
      - name: Setup SSH key
        run: |
          mkdir -p ~/.ssh
          echo "${{ secrets.SSH_PRIVATE_KEY }}" > ~/.ssh/id_rsa
          chmod 600 ~/.ssh/id_rsa
          ssh-keyscan -H ${{ secrets.MASTER_IP }} >> ~/.ssh/known_hosts
      - name: Save current deployment revision (pour rollback)
        run: |
          ssh ec2-user@${{ secrets.MASTER_IP }} \
          "kubectl get deployment myway-backend -n myway \
           -o jsonpath='{.metadata.resourceVersion}'" || true
      - name: Inject Worker IP in Frontend Manifest
        run: |
          sed -i "s/63.34.10.107/${{ secrets.WORKER_IP }}/g" \
            k8s/frontend-deployment.yml
      - name: Copy Kubernetes manifests to master node
        run: |
          scp -r k8s/ ec2-user@${{ secrets.MASTER_IP }}:/tmp/k8s-myway/
      - name: Apply Kubernetes manifests
        run: |
          ssh ec2-user@${{ secrets.MASTER_IP }} \
          "kubectl apply -f /tmp/k8s-myway/"
      - name: Wait for rollout - Backend
```

```
        run: |
          ssh ec2-user@${{ secrets.MASTER_IP }} \
          "kubectl rollout status deployment/myway-backend \
           -n myway --timeout=120s"
      - name: Wait for rollout - Frontend
        run: |
          ssh ec2-user@${{ secrets.MASTER_IP }} \
          "kubectl rollout status deployment/myway-frontend \
           -n myway --timeout=120s"
      - name: Auto Rollback on failure
        if: failure()
        run: |
          ssh ec2-user@${{ secrets.MASTER_IP }} \
          "kubectl rollout undo deployment/myway-backend -n myway; \
           kubectl rollout undo deployment/myway-frontend -n myway"
      - name: Verify deployment
        run: |
          ssh ec2-user@${{ secrets.MASTER_IP }} \
          "kubectl get pods -n myway && kubectl get svc -n myway"
```

Listing 6.1 – .github/workflows/cicd.yml (structure complète)

6.3 Détail des jobs du pipeline

6.3.1 Job 1 : Tests Backend et Frontend

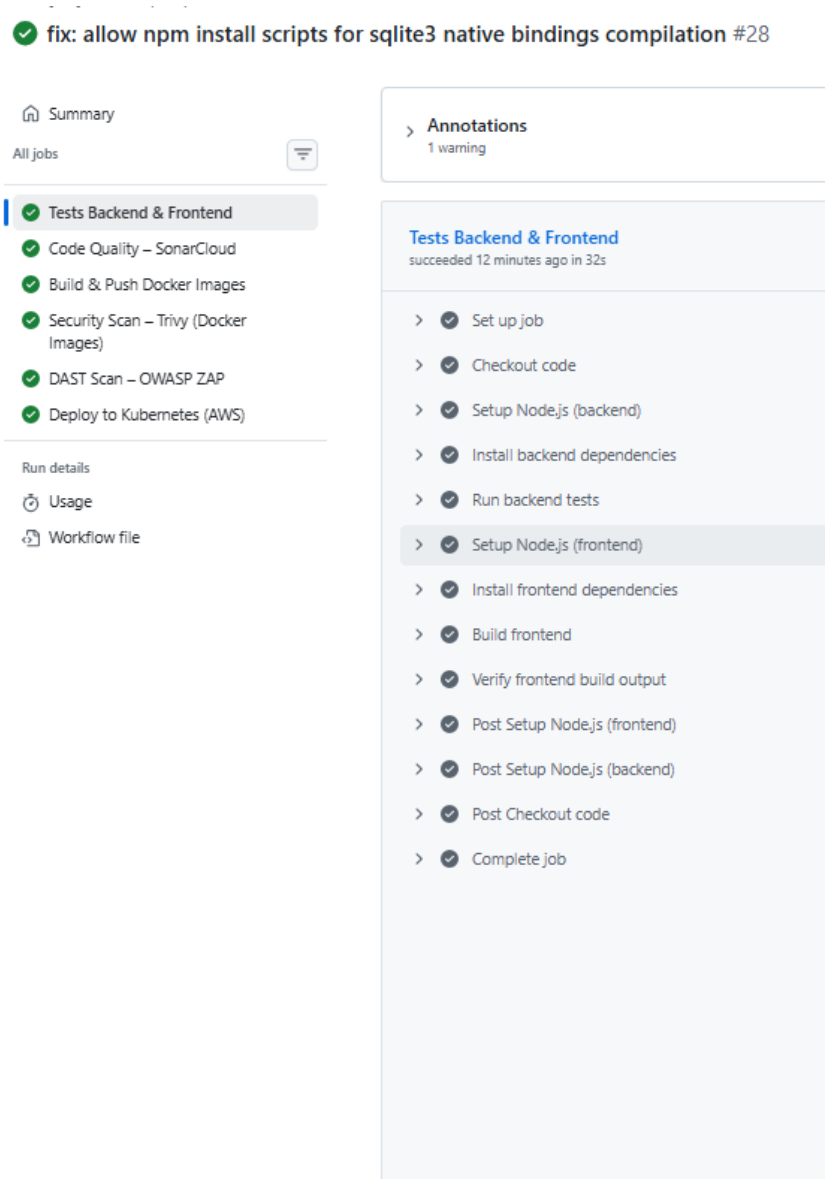

FIGURE 6.2 – Job 1 – Tests backend et build frontend réussis en 32s

6.3.2 Job 2 : Analyse de qualité SonarCloud

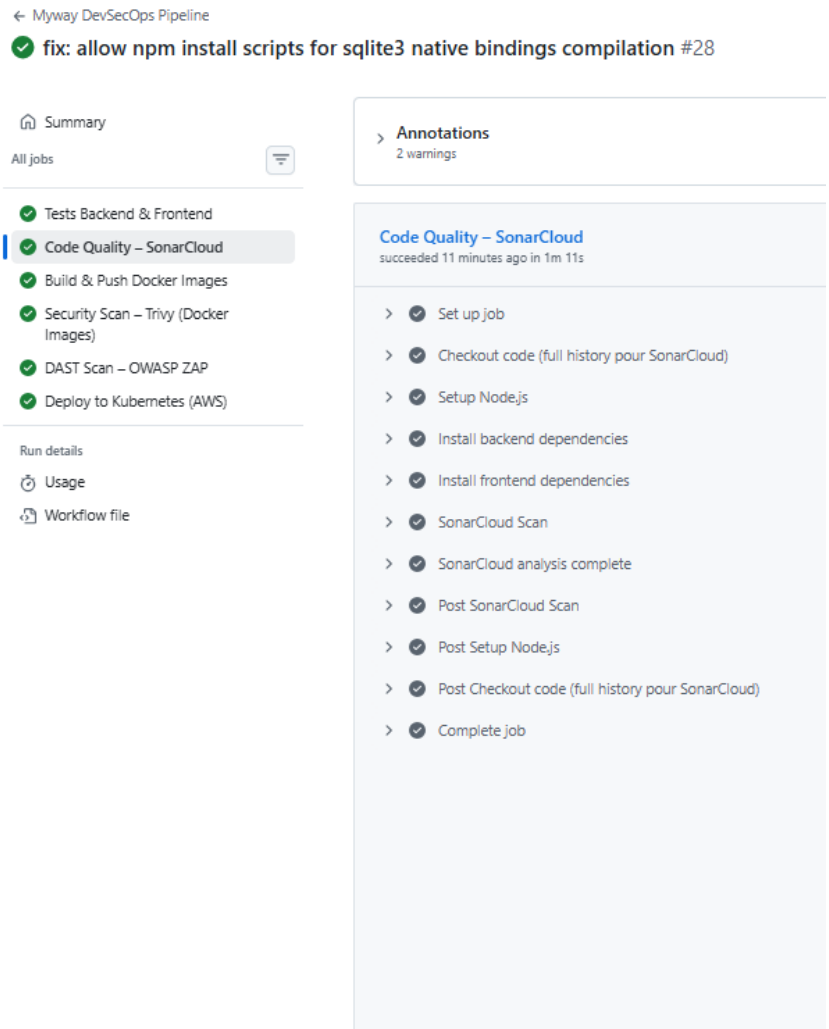

FIGURE 6.3 – Job 2 – Analyse SonarCloud réussie en 1m 11s

SonarCloud analyse l'ensemble du code source pour détecter les bugs, vulnérabilités et code smells. L'intégration utilise l'historique complet Git (`fetch-depth: 0`) pour une analyse précise.

6.3.3 Job 3 : Build et Push des images Docker

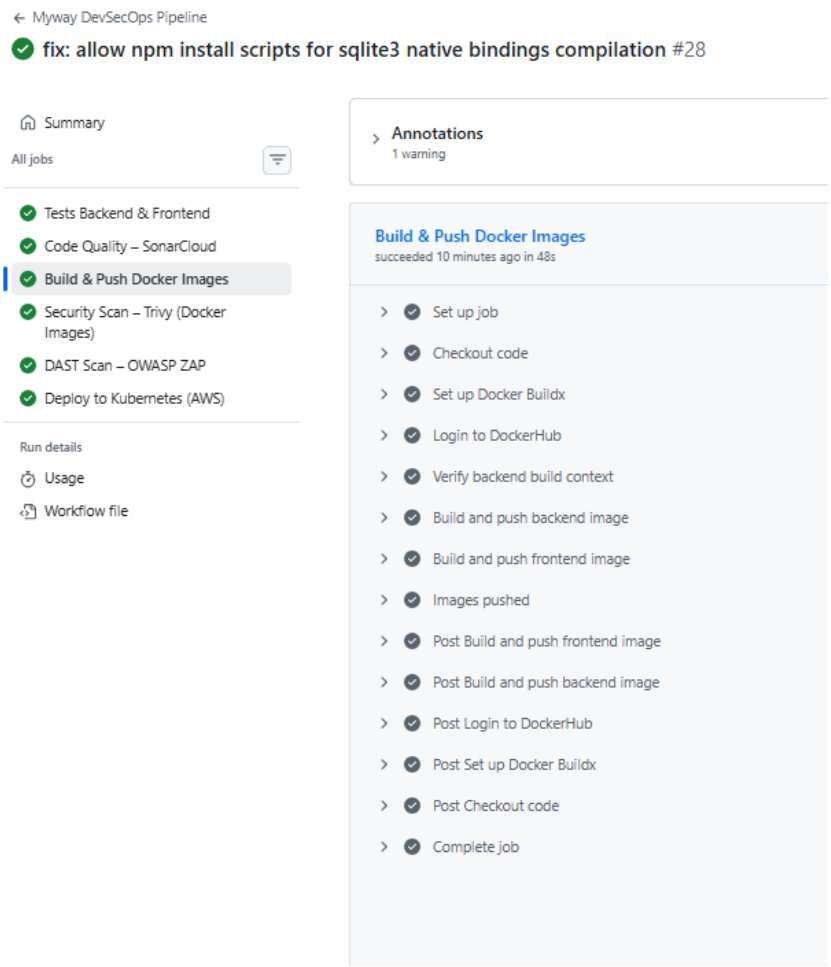

FIGURE 6.4 – Job 3 – Images Docker buildées et pushées sur DockerHub en 48s

6.3.4 Job 4 : Scan de sécurité Trivy

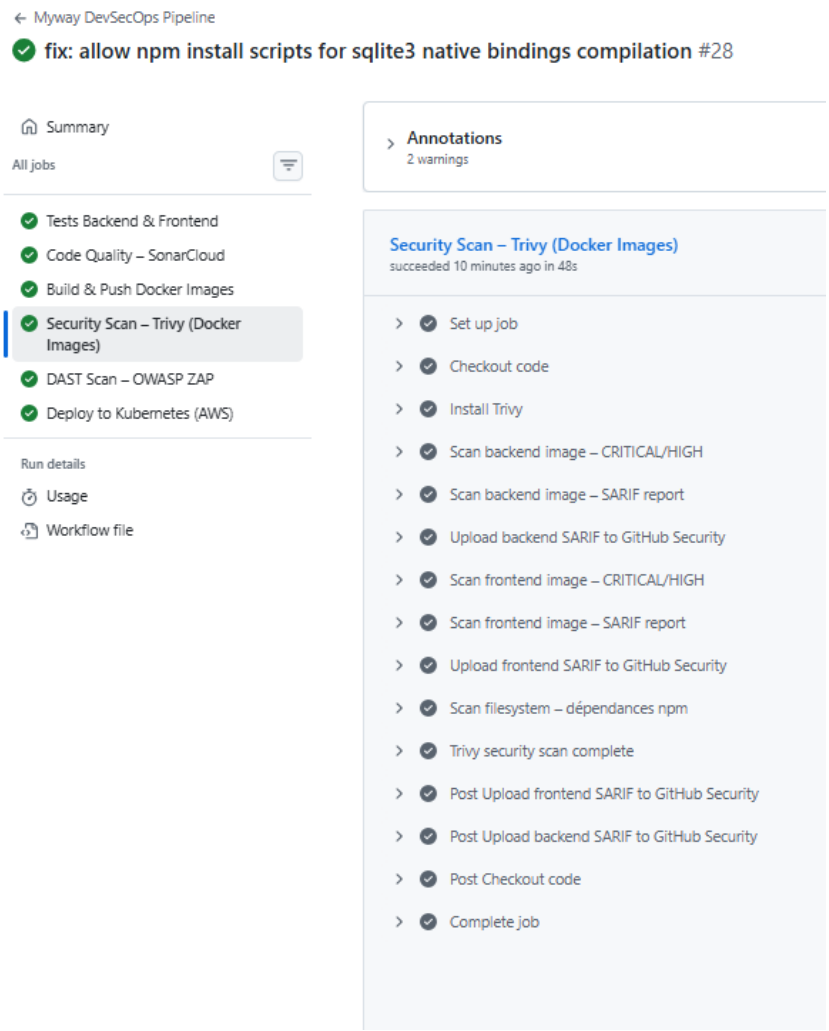

FIGURE 6.5 – Job 4 – Scan Trivy des images Docker et du système de fichiers en 48s

Trivy effectue trois analyses : scan CVE CRITICAL/HIGH du backend, scan CVE du frontend, et scan des dépendances npm. Les rapports SARIF sont uploadés vers GitHub Security.

6.3.5 Job 5 : DAST avec OWASP ZAP

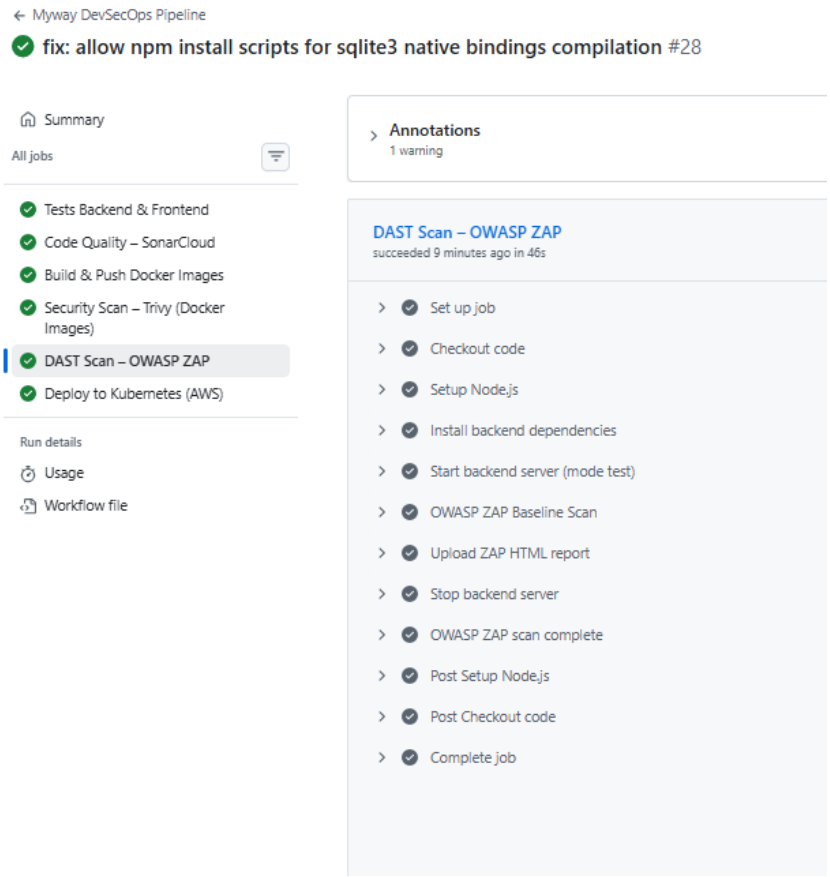

FIGURE 6.6 – Job 5 – Scan DAST OWASP ZAP sur l'API backend en 46s

6.3.6 Job 6 : Déploiement sur Kubernetes (AWS)

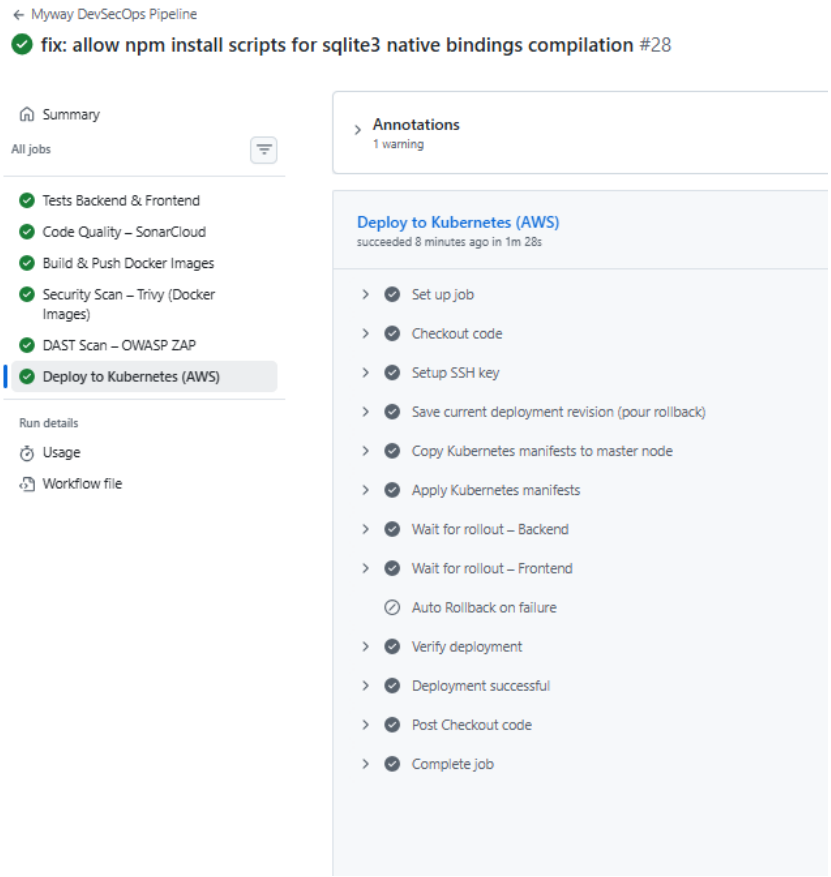

FIGURE 6.7 – Job 6 – Déploiement Kubernetes réussi en 1m 28s, rollback non déclenché

Le déploiement se fait via SSH sur le noeud maître. Avant la copie des manifests, l'étape *Inject Worker IP* substitue l'adresse IP réelle du worker dans `frontend-deployment.yml` via `sed`. Un mécanisme de **rollback automatique** annule le déploiement en cas d'échec du rollout.

6.4 Secrets GitHub Actions configurés

Table 6.1 – Secrets GitHub Actions du projet Myway

Nom du secret	Description
`DOCKERHUB_USERNAME`	Nom d'utilisateur DockerHub (`kappa20`)
`DOCKERHUB_TOKEN`	Token d'accès DockerHub
`SSH_PRIVATE_KEY`	Clé privée SSH pour accéder aux instances EC2
`MASTER_IP`	Adresse IP publique du noeud maître
`WORKER_IP`	Adresse IP publique du noeud worker (injectée dans le manifest frontend via `sed`)
`SONAR_TOKEN`	Token d'authentification SonarCloud

6.5 Durée et performances du pipeline

Table 6.2 – Durée de chaque job du pipeline

Job	Durée	Dépendances
Tests Backend & Frontend	32s	–
Code Quality – SonarCloud	1m 11s	tests
Build & Push Docker Images	48s	code-quality
Security Scan – Trivy	48s	build
DAST Scan – OWASP ZAP	46s	build
Deploy to Kubernetes (AWS)	1m 28s	security-scan, dast-scan

Les jobs Trivy et OWASP ZAP s'exécutent en parallèle après le build Docker. Le déploiement n'est lancé que si les deux scans de sécurité réussissent, garantissant qu'aucune image vulnérable critique n'atteint le cluster de production.

Chapitre 7

Monitoring avec Prometheus et Grafana

7.1 Architecture du monitoring

Table 7.1 – Composants du monitoring

Composant	Port	Rôle
Prometheus	30090 (NodePort)	Collecte et stockage des métriques
Grafana	30030 (NodePort)	Visualisation et tableaux de bord
Node Exporter	9100	Métriques système (CPU, RAM, disque)

7.2 Prometheus

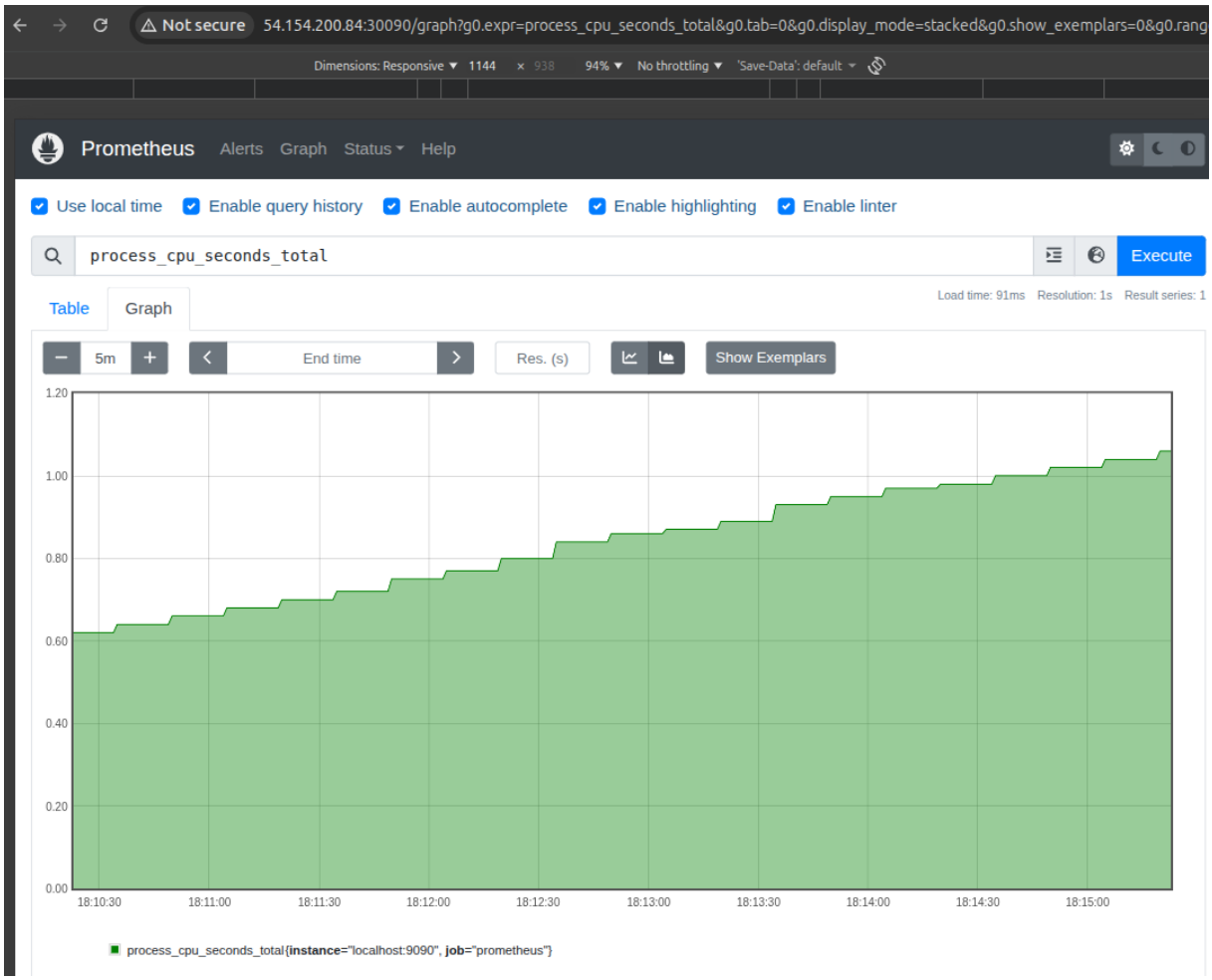

FIGURE 7.1 – Interface Prometheus – métrique process_cpu_seconds_total

7.3 Grafana

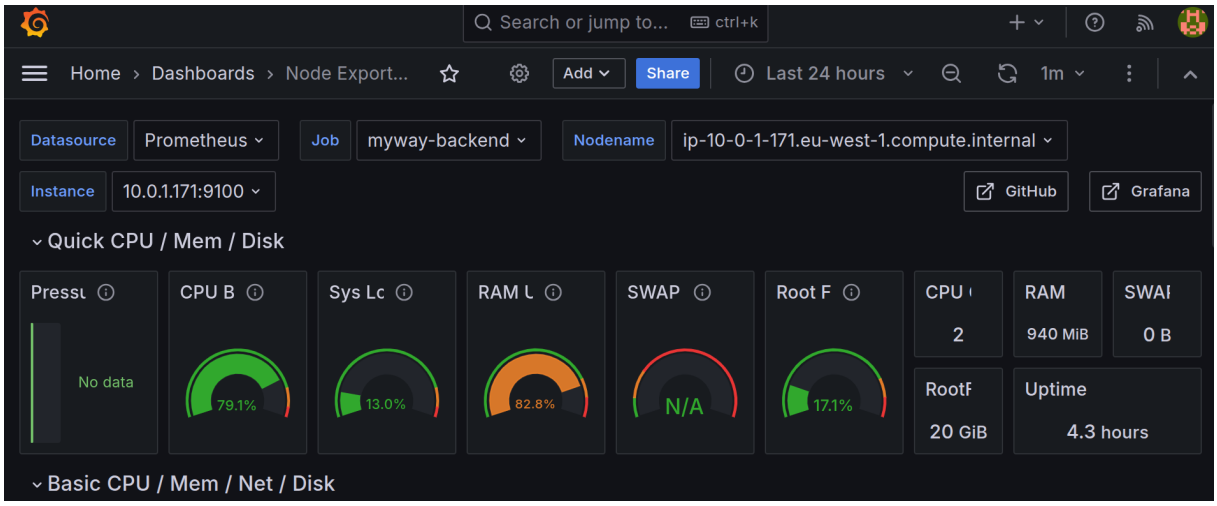

FIGURE 7.2 – Tableau de bord Grafana – Node Exporter : métriques système du backend Myway

TABLE 7.2 – Métriques système observées dans Grafana

Métrique	Valeur	État
CPU busy	79.1%	Élevé (charge normale)
System load	13.0%	Normal
RAM used	82.8%	Élevé (instances t3.micro)
Root filesystem	17.1%	Normal
CPU count	2	t3.micro
RAM total	940 MiB	t3.micro
Disk total	20 GiB	Volume EBS
Uptime	4.3 heures	Normal

Observation

L'utilisation RAM de 82.8% est attendue pour des instances `t3.micro` (1 Go de RAM) hébergeant à la fois les composants Kubernetes et les pods applicatifs. Pour une production réelle, des instances `t3.small` ou supérieures seraient recommandées.

Chapitre 8

Guide de Reproduction de l'Infrastructure

8.1 Prérequis

Table 8.1 – Outils à installer localement

Outil	Version	Installation
Terraform	v1.7.0+	`wget` depuis releases.hashicorp.com
Ansible	core 2.19+	`pip3 install ansible`
AWS CLI	v2.34+	Installateur officiel AWS
kubectl	v1.35+	Binaire officiel Kubernetes
Docker	24+	docs.docker.com
Git	2.x	Gestionnaire de paquets

8.2 Étapes de déploiement

8.2.1 Étape 1 : Cloner le dépôt

```
git clone https://github.com/kappa20/Myway.git && cd Myway
```

8.2.2 Étape 2 : Configurer AWS

```
aws configure
ssh-keygen -t rsa -b 4096 -f ssh/myway-key -N ""
```

8.2.3 Étape 3 : Déployer l'infrastructure Terraform

```
cd terraform/
terraform init && terraform plan && terraform apply
terraform output  # noter master_public_ip et worker_public_ip
```

8.2.4 Étape 4 : Allouer et associer les Elastic IPs

```
# Console AWS EC2 > Elastic IPs :
# 1. Allouer 2 Elastic IPs
# 2. Associer à k8s-master et k8s-worker
# 3. Mettre à jour ansible/inventory.ini
```

8.2.5 Étape 5 : Configurer et exécuter les playbooks Ansible

```
cd ansible/
ansible all -m ping  # vérifier la connectivité

# Executer les playbooks dans l'ordre
ansible-playbook playbooks/01-install-docker.yml
ansible-playbook playbooks/02-install-kubernetes.yml
ansible-playbook playbooks/03-init-master.yml
ansible-playbook playbooks/04-join-worker.yml
ansible-playbook playbooks/05-install-monitoring.yml
ansible-playbook playbooks/06-install-dashboard.yml

# Vérifier l'état du cluster
ssh -i ../ssh/myway-key ec2-user@<MASTER_IP>
kubectl get nodes
```

8.2.6 Étape 6 : Configurer les secrets GitHub Actions

Dans **Settings > Secrets and variables > Actions** du dépôt GitHub, ajouter :

- `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`
- `SSH_PRIVATE_KEY` : contenu de `ssh/myway-key`
- `MASTER_IP`, `WORKER_IP` : IPs élastiques des instances
- `SONAR_TOKEN` : token SonarCloud du projet

8.2.7 Étape 7 : Déclencher le pipeline CI/CD

```
git add . && git commit -m "feat: deploy infrastructure"
git push origin main
# => GitHub Actions démarre automatiquement
```

8.2.8 Étape 8 : Accéder au Monitoring et au Dashboard

```
# Les services ont été déployés automatiquement par Ansible (Playbooks 5
    et 6).
# Accès via le navigateur :
# Prometheus : http://<MASTER_IP>:30090
# Grafana    : http://<MASTER_IP>:30030 (identifiants par défaut)
```

8.2.9 Nettoyage des ressources AWS

```
aws ec2 stop-instances --instance-ids <master-id> <worker-id>
# Ou : cd terraform/ && terraform destroy
```

Chapitre 9

Conclusion

9.1 Conformité au cahier des charges

Table 9.1 – Grille de conformité au cahier des charges

Exigence CDC	Statut	Réalisation
Application REST conteneurisée	**OK**	Node.js/Express + React, Dockerfiles
Outil CI/CD	**OK**	GitHub Actions (6 jobs DevSecOps)
Build, test, déploiement automatisés	**OK**	Pipeline déclenché à chaque `git push`
Cluster Kubernetes (kubeadm)	**OK**	1 master + 1 worker, Flannel CNI, v1.29.0
Déploiement K8s	**OK**	Manifests YAML, services NodePort et **Auto-scaling (HPA)**
Terraform (minimum 2 VMs)	**OK**	2 instances EC2 t3.micro, VPC, Elastic IPs
Ansible (configuration des noeuds)	**OK**	6 playbooks : Docker, K8s, init master, join worker
Diagramme d'architecture	**OK**	Chapitre 2
Code source + Dockerfile	**OK**	Dépôt public kappa20/Myway sur GitHub
Manifests Kubernetes YAML	**OK**	Dossier `k8s/` : namespace, deployments, services
Monitoring Prometheus/Grafana	**OK**	Prometheus (30090) + Grafana (30030)
Documentation reproductible	**OK**	Chapitre 8 : guide en 8 étapes

Synthèse de conformité

L'ensemble des livrables exigés dans le cahier des charges ont été implémentés et testés. En complément des objectifs obligatoires, l'équipe a choisi d'intégrer des outils d'analyse de qualité et de sécurité (SonarCloud, Trivy, OWASP ZAP) afin de se rapprocher d'une démarche DevSecOps réaliste. Ces ajouts ont nécessité un travail de débogage conséquent, notamment pour la compilation des binaires natifs sqlite3 dans les images Docker et la configuration des runners GitHub Actions.

9.2 Bilan du projet

Ce projet nous a permis de mettre en pratique, sur une infrastructure cloud réelle, les principes fondamentaux du mouvement DevOps. La principale valeur ajoutée de ce travail réside moins dans la complexité de l'application elle-même que dans la rigueur de la chaîne de déploiement mise en place autour d'elle.

Parmi les points qui ont requis le plus d'efforts de débogage :

— La compilation des binaires natifs de sqlite3 dans les images Docker Alpine, qui nécessitent un compilateur C disponible. La solution finale a consisté à ajouter `RUN apk add -no-cache python3 make g++` dans le Dockerfile avant l'installation des dépendances, et à utiliser `npm install -production` sans flag `npm install -production` sans flag `-ignore-scripts`).

— L'initialisation du cluster kubeadm sur des instances EC2 avec swap désactivé et paramètres sysctl correctement configurés.

— La gestion des adresses IP publiques variables à chaque recréation de l'infrastructure (partiellement résolue par les Elastic IPs AWS).

— L'intégration d'OWASP ZAP dans un runner GitHub Actions sans serveur dédié (contournée en démarrant le backend en mode test dans le runner).

9.3 Compétences acquises

- **Infrastructure as Code :** maîtrise de Terraform pour le provisionnement cloud reproductible.
- **Automatisation :** écriture de playbooks Ansible pour le bootstrapping d'un cluster Kubernetes.
- **Orchestration :** déploiement et gestion de workloads Kubernetes (Deployments, Services, Namespaces).
- **CI/CD :** conception d'un pipeline GitHub Actions intégrant qualité et sécurité.
- **DevSecOps :** intégration de SonarCloud, Trivy et OWASP ZAP dans la chaîne de livraison.
- **Observabilité :** configuration de Prometheus et Grafana pour le monitoring système.

9.4 Limites techniques identifiées

Persistance des données SQLite. Le volume `emptyDir` est lié au cycle de vie du pod. Tout redéploiement entraîne la perte des données. La solution serait un `PersistentVolumeClaim` sur AWS EBS ou une migration vers AWS RDS.

Injection de VITE_API_URL. L'approche `sed` dans le pipeline reste fragile face aux changements d'IP. Un Ingress Controller avec nom de domaine fixe serait plus robuste.

Ressources limitées des instances t3.micro. L'utilisation mémoire dépasse 80%. Des instances `t3.small` (2 Go) seraient le minimum recommandé en production.

Absence de TLS. Tous les services sont exposés en HTTP. Une configuration HTTPS avec cert-manager et Let's Encrypt serait nécessaire en contexte réel.

9.5 Perspectives d'amélioration

- Migration vers PostgreSQL sur AWS RDS pour remplacer SQLite3.
- Ingress Controller NGINX avec TLS/HTTPS via Let's Encrypt.
- Cluster Kubernetes managé (Amazon EKS) pour la haute disponibilité.
- Ajout de Loki et Promtail pour la centralisation des logs applicatifs.
- Déploiement blue/green ou canary pour des mises à jour sans interruption.

Bibliographie

[1] The Kubernetes Authors, *kubeadm init – Reference*, 2025. https://kubernetes.io/docs/reference/setup-tools/kubeadm/kubeadm-init/

[2] The Kubernetes Authors, *Service – NodePort*, 2025. https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport

[3] The Kubernetes Authors, *Volumes – emptyDir*, 2025. https://kubernetes.io/docs/concepts/storage/volumes/#emptydir

[4] Flannel Authors, *Flannel – Container Network Interface*, GitHub, 2025. https://github.com/flannel-io/flannel

[5] HashiCorp, *Terraform AWS Provider – aws_instance & aws_eip*, 2025. https://registry.terraform.io/providers/hashicorp/aws/latest/docs

[6] Red Hat, *Ansible – ansible.builtin.yum module*, 2025. https://docs.ansible.com/ansible/latest/collections/ansible/builtin/yum_module.html

[7] GitHub, *Using secrets in GitHub Actions*, 2025. https://docs.github.com/en/actions/security-guides/using-secrets-in-github-actions

[8] Aqua Security, *Trivy – GitHub Actions Integration*, 2025. https://aquasecurity.github.io/trivy/latest/tutorials/integrations/github-actions/

[9] OWASP Foundation, *ZAP – Baseline Scan GitHub Action*, 2025. https://www.zaproxy.org/docs/docker/github-actions/

[10] SonarSource, *SonarCloud – GitHub Actions Integration*, 2025. https://docs.sonarcloud.io/advanced-setup/ci-based-analysis/github-actions-for-sonarcloud/

[11] Grafana Labs, *Node Exporter Full – Dashboard 1860*, 2025. https://grafana.com/grafana/dashboards/1860-node-exporter-full/

[12] Docker Inc., *Multi-stage builds – React avec Nginx*, 2025. https://docs.docker.com/build/building/multi-stage/